

UD2. Administración de Linux (Despliegue de Aplicaciones con Contenedores)



Q Autor: Fernando Barcina con copilot

Ahora que ya hemos terminado el primer tema introductorio, vamos a pasar a afianzar en este tema, un repaso de todos los conceptos de Linux que verás de manera directa o indirecta posteriormente para el control de contenedores. Principalmente trabajaremos:

- ❑ Montaje y gestión de ficheros: básico si luego queremos hacer cosas más complejas como presentar carpetas desde el host al contenedor;
- ❑ gestión de usuarios, grupos y permisos de acceso;
- ❑ interconexión y configuración de red: no olvidemos que muchos de nuestros contenedores se accederán por red en un puerto concreto;
- ❑ monitorización de procesos y registros del sistema: también inspeccionaremos contenedores pero ¿cómo inspeccionar si algo falla en el sistema Linux que soporta esos contenedores? ¿o en la parte Linux del contenedor?

1. Sistema de archivos

Objetivo

Vamos a estudiar en este apartado uno de los elementos que más caracterizan a un SO, ya que sobre el **sistema de archivos** se monta toda la estructura que permite al sistema **almacenar, manipular,**

organizar, acceder y consultar los datos que están guardados en archivos.

Cada **sistema de archivos (file system)** utiliza métodos diferentes para llevar a cabo las operaciones mencionadas anteriormente, aunque por otra parte todos los sistemas de archivos tienen características comunes. Por ejemplo todos los sistemas de archivos actuales utilizan los **directorios** o **carpetas** para organizar a los archivos.

¿Qué sistema de archivos tiene la Máquina Virtual Ubuntu que instalaste en el anterior tema?

Aunque durante la instalación de Ubuntu el asistente básico de instalación nos guió durante el proceso y es muy probable que no necesitemos saber nada más, como desarrollador en sistemas Linux sí te vendrá bien saber:

- qué tipo de sistemas de archivos se utilizan para formatear las particiones y montarlas
- qué estructura típica de carpetas dispone una distro basada en Linux

[Gparted](#)  es una aplicación GNU/Linux que permite gestionar discos duros y particiones. Con ella puedes ver las unidades de disco que tienes instaladas en tu ordenador, comprobar las particiones, acceder a información diversa sobre cada partición, como su sistema de archivos, el tamaño total o la disposición física del contenido en el disco. Asimismo, permite modificar el tamaño de las particiones de varios tipos, entre otros, **NTFS, FAT y Ext4**. En definitiva, te permitirá crear, formatear, redimensionar, mover y borrar cualquier partición instalada en tu disco.

Como se ha indicado anteriormente, existen distintos tipos de sistemas de archivos, siendo los siguientes los más utilizados para entornos mono y multiusuario:

Sistema de archivos	S.O. soportados	Tamaño máx. de partición/vol.	Tamaño máx. de fichero	Unidades lógicas o árbol de directorios
FAT32	Linux, Windows (menos el W95 y NT 4.0)	2 TB	4GB 	A:, B:, C: ..., Z:
NTFS	Windows con tecnología NT (menos 9X y ME) y Linux	256 TB	16 TB	A:, B:, C: ..., Z: o un solo árbol de directorios
ext4	GNU – Linux	1 EB	16 TB	Árbol de directorios, pej. /dev/sda1

Nosotros en Ubuntu, utilizaremos [ext4](#)  que además de **no tener prácticamente fragmentación** es **transaccional** (en inglés journaling: también se le conoce como «registro por diario»), es decir, puede implementar **transacciones**. Se basa en llevar un *journal* o registro de diario en el que se almacena la información necesaria para restablecer los datos afectados por la transacción en caso de que ésta falle.

En nuestro caso cuando el sistema de archivos necesita realizar una operación de disco, se guardan en un registro los cambios que se harán en la estructura del directorio, los bloques de disco que se verán afectados y los cambios que se realizarán en los archivos implicados. De esta forma cuando se produce una avería, un cuelgue del sistema o un fallo de alimentación eléctrica, se podrán deshacer los cambios que se hayan realizado si no termina de forma satisfactoria la operación completa... esto suele pasar con aplicaciones ofimáticas cuando te preguntan si quieres recuperar un documento que se quedó con información a medio grabar o inconsistente.

El etiquetado de los discos en sistemas GNU/Linux

En los sistemas Windows cada dispositivo (incluyendo las particiones del disco) tiene asignada una letra. Así, la primera partición del disco duro es C: (donde se instala el sistema operativo y las aplicaciones, de forma habitual), la segunda partición del disco duro es D:, el lector de CD es E:, y así sucesiva y alfabéticamente si se instalan otros dispositivos, como discos duros externos o memorias USB. Esto trae como consecuencia que, cada vez que quieres acceder a un dispositivo distinto, tienes que volver a Mi PC y/o adivinar qué letra le ha puesto el sistema a tu dispositivo. ¿Acaso no lo has notado cuando has querido extraer de forma segura alguno de los USB conectados? ¿Cuál era: el G o el H?

GNU/Linux utiliza una organización de la información más adecuada y coherente. En Linux todo son ficheros... ¡y todo es todo!: un lector de CD es un fichero, un USB es también un fichero, y una partición de un disco duro ídem de ídem. Y utiliza una terminología mucho más lógica.

Linux incluye a los dispositivos dentro de la propia estructura de ficheros, de tal forma que **los dispositivos se encuentran todos "colgando" del directorio "/dev"**, utilizando la siguiente nomenclatura para identificar a los distintos dispositivos y particiones de nuestra máquina:

En general, para los antiguos discos duros **SCSI o SATA o SAS** es **/dev/sdXY** donde **X** es una letra que indica la situación del disco duro en el BUS e **Y** es el número de partición. Así, Ubuntu nombra los archivos de acuerdo con el esquema determinado que te mostramos a continuación:

- /dev/sda es el primer disco duro identificado
- /dev/sdb es el segundo disco duro identificado

si tienes más discos duros en tu ordenador Ubuntu los denominará sdc, sdd y así sucesivamente.

- /dev/sda1 es la partición primaria en el disco sda

las otras tres particiones primarias en ese disco, si se trata de un particionado antiguo o MBR, serán sda2, sda3 y sda4, y en caso de ser lógicas serán a partir de la 5, es decir: sda5 primera partición lógica, sda6: segunda partición lógica, y así sucesivamente. En cualquier caso, esto se está convirtiendo en historia con [EFI/GPT](#)  que para hacer pleno uso de discos grandes (2TB y más) se puede tener hasta 128 particiones por defecto (sin tener en cuenta si son primarias o lógicas).

Y este mismo patrón sigue para todos los dispositivos que conectas al equipo. Así, si conectas un dispositivo USB o un disco duro externo en un sistema Ubuntu, estos dispositivos serán reconocidos como /dev/sdb (si sólo tienes un disco duro en el ordenador) o sucesivos, y serán accesibles desde /media con la etiqueta que tuviesen asociada al ser formateados (por ejemplo, /media/KINGSTON). ¿No te parece esto ahora más lógico que lo que has manejado hasta ahora en sistemas Windows?

Si ya tienes Ubuntu instalado abre una consola o terminal y escribe la orden

```
sudo fdisk -l
```

Este comando te permite ver una visión general de los discos duros disponibles y los medios de almacenamiento USB conectados incluyendo además las particiones y sistemas de ficheros que contienen. Lo veremos en más detalle en el siguiente apartado.

1.1. Gestores de particiones

En Linux el programa más habitual encargado de gestionar las particiones es el fdisk. Este posee una interfaz texto que permite crear, modificar y borrar particiones de casi cualquier tipo escribiendo la tabla de particiones en el sector 0 (conocido como superblock). Funciona en modo interactivo y para ejecutarlo se le pasa como argumento el disco duro a particionar a través del dispositivo correspondiente. Ejemplo:

```
# fdisk /dev/sda
```

Desde el prompt que muestra fdisk es posible ejecutar comandos que permiten realizar las distintas operaciones. Algunos de estos comandos se listan a continuación:

- m : muestra la ayuda de todos los posibles comandos.
- p : imprime la tabla de particiones del disco duro actual. Su salida es similar a la del siguiente ejemplo:
- n : añade una nueva partición. Para ello se debe indicar si esta será lógica o primaria, el sector inicial (se ofrece uno por defecto) y el sector final o el tamaño total de la partición. Por defecto las particiones se crean del tipo Linux.
- d : permite borrar una partición.
- l : lista todos los tipos de particiones reconocidas por fdisk. Los tipos se identifican utilizando números hexadecimales.
- t : permite cambiar el tipo de una partición.
- w : actualiza la tabla de particiones y termina. Hasta que no se ejecute este comando los cambios realizados no tendrán validez.
- q : permite salir sin salvar los cambios realizados hasta el momento.

```
usuario@debian6:~$ sudo fdisk -l
Disco /dev/sda: 16.1 GB, 16106127360 bytes
255 heads, 63 sectors/track, 1958 cylinders
Units = cilindros of 16065 * 512 = 8225280 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disk identifier: 0x000a5888

Disposit. Inicio Comienzo Fin Bloques Id Sistema
/dev/sda1 * 1 1873 15037440 83 Linux
/dev/sda2 1873 1958 688128 5 Extendida
/dev/sda5 1873 1958 688128 82 Linux swap / Solaris
usuario@debian6:~$
```

Un uso habitual que hacemos del fdisk es utilizarlo para listar todos los dispositivos de almacenamiento del sistema, a fin de poder montarlos. Esto lo conseguimos con la opción -l (ele) y sin indicar ningún dispositivo.

Existen otros programas particionadores en Linux, tanto pensados para ejecutarlos desde terminal como para ser lanzados desde el entorno gráfico. Algunos de los más usados son: cfdisk, gparted, qtparted, etc.

Ubuntu Server | Particionado de discos | fdisk, ...



*Particionado de discos | fdisk, mkfs y
mount: <https://youtu.be/wPngAcPjc9M>*



Ubuntu Server | Particionado de discos | Gparted



🔍 *Particionado de discos | Gparted: <https://youtu.be/7nPksA-yijs>*



Montaje y desmontaje

En Linux los dispositivos físicos de la máquina en general y los de almacenamiento de información, en particular, son manipulados a través de ficheros especiales ubicados en el directorio /dev. Los discos duros, las particiones de éstos, las unidades de CD-DVD son ejemplos de estos dispositivos con los cuales

interactuamos constantemente. Pero trabajar directamente sobre los dispositivos usualmente se incorporan al sistema de ficheros tradicional.

Esta acción se conoce como **"montar"**, que en definitiva es asociar el dispositivo a un directorio determinado para hacerlo accesible desde éste.

Como se explicó anteriormente las particiones de los discos en Linux se montan en directorios como /, /home y /usr. El sistema tiene un fichero llamado **/etc/fstab** en el cual se especifican dónde y en qué forma se montan los diferentes dispositivos. Veamos un ejemplo de dicho fichero:

```
delarioja@ubuntu100-SOM:~$ cat /etc/fstab
# /etc/fstab: static file system information.
#
# Use 'blkid' to print the universally unique identifier for a
# device; this may be used with UUID= as a more robust way to name devices
# that works even if disks are added and removed. See fstab(5).
#
# <file system> <mount point> <type> <options> <dump> <pass>
proc /proc proc nodev,noexec,nosuid 0 0
# / was on /dev/sda1 during installation
UUID=aca22aed-b33c-4b50-b6a1-cc374b92f382 / ext4 errors=remount-ro 0 1
#/dev/sda1 / ext4 errors=remount-ro 0 1
# swap was on /dev/sda5 during installation
UUID=03e31199-937a-4f9f-9287-de559bf330a6 none swap sw 0 0
/dev/fd0 /media/floppy0 auto rw,user,noauto,exec,utf8 0 0
```

La línea amarilla y la verde harían lo mismo; la que tiene una # al principio no será tenida en cuenta

La

estructura de las instrucciones es de **6 columnas separadas por espacios o tabuladores**:

<dispositivo> <punto_de_montaje> <sistema_de_archivos> <opciones> <dump-frecuencia> <pass>
Cada línea en este fichero describe un dispositivo, indicando los siguientes aspectos para cada uno:

- ❑ Nombre del dispositivo o etiqueta. Ejemplos: /dev/sda1, /dev/sdc2... También puede aparecer codificada como **UUID**.
- ❑ Directorio donde se monta. Ejemplos: /, /home, /mnt/usb, /var/www/html, etc.
- ❑ Sistema de ficheros. Ejemplos: ext4, ntfs, nfs, swap, iso9660, auto, etc.
- ❑ Opciones de montaje. Ejemplos: ro, rw, exec, auto, user, etc.
- ❑ Dos valores numéricos: el primero toma los valores 0/1 indicando si al dispositivo se le hará **dump** (especie de backup/copia de seguridad) o no (si es cero no se toma en cuenta ese dispositivo). El segundo número indica el orden en que la aplicación **fsck** revisará la partición en busca de errores durante el inicio, si es cero el dispositivo no se revisa. Para el sistema de ficheros raíz / se suele poner un 1 de modo que el comando los chequeará primero, al resto se le puede poner un 2 para hacerlos posteriormente a los de valor 1.

Las opciones de montaje son numerosas. Las más usadas se listan a continuación:

- ❑ **auto**: indica que el dispositivo se monta siempre que se inicie el sistema. La opuesta es **noauto**.
- ❑ **rw**: indica que el dispositivo se monta con permisos de lectura y escritura.
- ❑ **ro**: indica que el dispositivo se monta con permisos de lectura solamente.
- ❑ **user**: indica que cualquier usuario puede montar y solo el mismo usuario podrá desmontar el dispositivo. La opción opuesta es **nouser**.
- ❑ **exec**: indica que los binarios ejecutables almacenados en el dispositivo se pueden ejecutar. La opción opuesta es **noexec**.
- ❑ **async**: expresa que todas las operaciones de entrada y salida se hacen de forma asíncrona, o sea, no necesariamente en el momento en que se invocan. La opción opuesta es **sync**.
- ❑ **defaults**: es una opción equivalente a la unión de **rw, exec, auto, nouser, async** -> **RECOMENDADA**
- ❑ **netdev**: no intenta montar el dispositivo si la red no está habilitada

Ejemplos de líneas en el fichero /etc/fstab:

/dev/sda2	/home	ext4	defaults	1	2
/dev/cdrom	/mnt	iso9660	noauto,owner,ro	0	0
/dev/sr0	/mnt/cdrom	auto	noauto,owner,ro	0	0
/dev/sda9	swap	swap	defaults	0	0
/dev/hdc1	/alien	ext2	ro,noexec,owner,noauto	1	0
/dev/hda1	/mnt/win	msdos	ro,async,nouser,auto	0	0

mount y umount

Si queremos que un dispositivo se monte automáticamente cada vez que el sistema se inicie, basta con colocar una línea apropiada en el fichero /etc/fstab

Si lo que queremos es montar o desmontar dispositivos directamente desde línea de comandos, podemos emplear los comandos mount y umount respectivamente. Éstos mantienen una lista de los dispositivos montados en el fichero /etc/mtab

Sintaxis:

mount [opciones] [dispositivo] [dir]

umount [opciones] <dir> | <dispositivo>

Algunas opciones:

- -a: en el caso de mount monta todos los dispositivos que tienen la opción auto en el fichero fstab, y para umount desmonta todo lo que está en el fichero /etc/mtab.
- -t <tipo>: indica el tipo de file system a montar

Cuando se especifican en el fichero fstab las características del montaje de un dispositivo, para montarlo no es necesario indicarlo todo, basta con poner el nombre del dispositivo o el directorio donde se monta por defecto.

Ejemplos:

\$ mount -a # monta todos los dispositivos con la opción auto en el fichero /etc/fstab (este comando se ejecuta al iniciar el sistema)

\$ mount /dev/sr0 /media/etiqueta # monta el dispositivo sr0 que a veces es el cdrom en el directorio /media/etiqueta

\$ mount /dev/cdrom # monta el cdrom. Toma las especificaciones del fichero /etc/fstab

\$ mount /mnt/cdrom # hace lo mismo que el anterior

\$ umount -a -t ntfs # desmonta todo los dispositivos con file system ntfs especificados en /etc/mtab

\$ umount /mnt/cdrom # desmonta el cdrom

\$ mount /dev/sda5 /home/usuario/llavero

\$ umount /dev/sda5 # desmonta el dispositivo anterior

\$ umount /home/usuario/llavero # otra manera de desmontar el dispositivo anterior

Siempre que un dispositivo esté siendo utilizado por el sistema no se podrá desmontar. Este emitirá un mensaje de error como en el siguiente ejemplo:

\$ umount /mnt/cdrom

umount: /mnt/cdrom: **device is busy**

Un **dispositivo puede estar ocupado** por el simple hecho de tener abierto un shell en el directorio donde se montó, haber lanzado un ejecutable al background desde ese directorio, o haber montado otro dispositivo en un subdirectorio del mismo.

Hoy en día la mayoría de las distribuciones cuentan con algún tipo de programa monitor en memoria que se encargará de montar automáticamente cualquier dispositivo externo que conectemos al sistema. En el caso de Debian por ejemplo, se encargan del auto montado los paquetes udisks y gnome-disk-utility.

df

Un comando muy útil en Linux es df. Este se emplea para conocer información acerca de las particiones y dispositivos montados actualmente en el sistema. Para cada dispositivo se muestra por defecto su tamaño,

el espacio empleado, que porcentaje esta empleado, así como el directorio donde se ha montado.

Sintaxis: `df [opciones] [directorio]`

Algunas opciones:

- `-h`: (human readable view) por defecto los tamaños se muestran en bytes y con esta opción se hace de forma más legible (Ej. G para gigabytes, M megabytes)

```
delarioja@ubuntu100-SOM:~$ df
```

S.ficheros	1K-bloques	Usados	Disponibles	Uso%	Montado en
/dev/sda1	29799484	4303332	23975768	16%	/
udev	504836	4	504832	1%	/dev
tmpfs	205228	788	204440	1%	/run
none	5120	0	5120	0%	/run/lock
none	513060	124	512936	1%	/run/shm
/dev/sr0	60934	60934	0	100%	/media/VMware Tools

mkfs

Se utiliza para crear un sistema de ficheros en un dispositivo (formatear a alto nivel). Por defecto el sistema de ficheros creado es del tipo ext4. Sirve de interfaz para otros comandos más específicos como `mkfs.ntfs`, `mkfs.fat`, `mkfs.ext4`

Ejemplos:

```
# mkfs /dev/sdb1
```



Tarea resuelta

1. Añade un disco nuevo a tu máquina virtual Ubuntu, formatea y monta la partición principal del mismo, por ejemplo (suponiendo que sea el segundo disco): `/dev/sdb1`
2. Modifica el fichero `/etc/fstab` para que se monte automáticamente al iniciar el sistema.

Mostrar retroalimentación

1.2. Directorios generales en distribuciones GNU/Linux

El **estándar de jerarquía del sistema de archivos** (o **FHS**, del inglés *Filesystem Hierarchy Standard*) es una norma que define los **directorios** principales y sus contenidos en el **sistema operativo GNU/Linux** y otros sistemas de la familia **Unix**. Se diseñó originalmente en 1994 para estandarizar el sistema de archivos de las **distribuciones de Linux**, basándose en la tradicional organización de directorios de los sistemas Unix.

Los directorios generales en distribuciones GNU/Linux son:

/: El directorio raíz. Es la base de la estructura del árbol del sistema de archivos. **Todos los demás archivos y directorios**, independientemente de su localización física, **son accesibles desde este directorio**.

/bin: Directorio de **comandos binarios** (se dice binario por estar guardados en código binario que se carga directamente en memoria, es lo mismo que **ejecutable** o **.exe** en Windows). Incluye los ejecutables **disponibles para todos los usuarios** que son parte del sistema operativo UNIX (**cp**, **mv**, **ls**, **rm**, etc.).

/sbin: **Ejecutables** para tareas de **administración** del sistema (**system binary**)

Directorio raíz de toda la jerarquía de archivos del sistema

/

Jerarquía primaria

/bin/	Comandos binarios esenciales de usuario
/boot/	Archivos estáticos del selector de arranque
/dev/	Archivos de unidades
/etc/	Configuración de sistema de Host específico Directorios requeridos: opt, X11, sgml, xml
/home/	Directorio 'home' de usuario
/lib/	Librerías esenciales compartidas y módulos de Kernel
/media/	Punto de montaje para medios removibles
/mnt/	Punto de montaje temporal para sistemas de archivos
/opt/	Agregados de paquetes de Software y Aplicaciones
/sbin/	Binarios de sistema
/srv/	Datos para los servicios provistos por este sistema
/tmp/	Archivos temporales
/usr/	Utilidades y aplicaciones de (Multi-)usuario Jerarquía secundaria Directorios requeridos: bin, include, lib, local, sbin, share
/var/	Variables de archivo
/root/	Directorio 'home' del usuario root
/proc/	Documentación del sistema de archivos virtual del Kernel y las condiciones de los procesos en archivos de texto

Jerarquía estándar de archivos del sistema

Traducido a español por Franco Ferrari
franco.ferrari@rincomovil.com
<http://eldebianita.hazlo-asi.biz/>

/boot: Ficheros utilizados durante el **arranque del sistema** (núcleo y configuración del gestor de arranque GRUB)

/dev: Directorio de **dispositivos** (devices en inglés), contiene archivos especiales como impresoras, discos, terminales, etc.

/etc: Archivos de **configuración** del sistema.

/home: Directorio con los **ficheros personales de los usuarios**, tales como Documentos, Descargas, Imágenes, etc. Contiene una subcarpeta por cada usuario creado en el sistema excepto

root. Sería como la carpeta C:\Usuarios de las últimas versiones de Windows.

/root: Directorio con los **ficheros personales del usuario root** o administrador del sistema.

/mnt: Directorio de montaje. Este directorio está vacío y se usa **para montar/acceder particiones de discos** temporalmente.

/media: En Ubuntu se usa para **montar de manera automática unidades o medios removibles**: pendrives, cdrom, etc.

/lib: Contiene los archivos de **librerías o subprogramas compartidos** por algunos programas del sistema (análogo a "C:\Windows\System32" en Windows).

/lost+found: Directorio de **archivos perdidos**. Errores de disco o un apagado incorrecto del sistema puede causar que los archivos acaben en esta carpeta después de un chequeo de disco (fsck).

/proc: Sistema de ficheros virtual que informa de los procesos y estados del núcleo. Contiene principalmente ficheros de texto con **información sobre el hardware reconocido en el sistema** como por ejemplo características del procesador, temperatura, etc.

/tmp: Disponible para todos los usuarios como directorio para archivos **temporales**.

/usr: Contiene **programas generados localmente, ejecutables** para los comandos de usuarios, documentación, etc. (Unix system resources). Vendría a ser como la carpeta "C:\Archivos de programa" de Windows.

/var: Ficheros **variables en tamaño** como son logs (o ficheros que guardan los sucesos que acontecen en el sistema), bases de datos, directorio raíz de servidores HTTP y FTP, colas de correo, etc. En los servidores tiene un carácter tan importante como /home en los ordenadores de escritorio, razón por la que ambos directorios se suelen montar en particiones independientes para, en caso de tener que reinstalar el sistema, no perder estos datos tan importantes.

Ubuntu | 6.- Estructura del sistema de ficheros o archi...

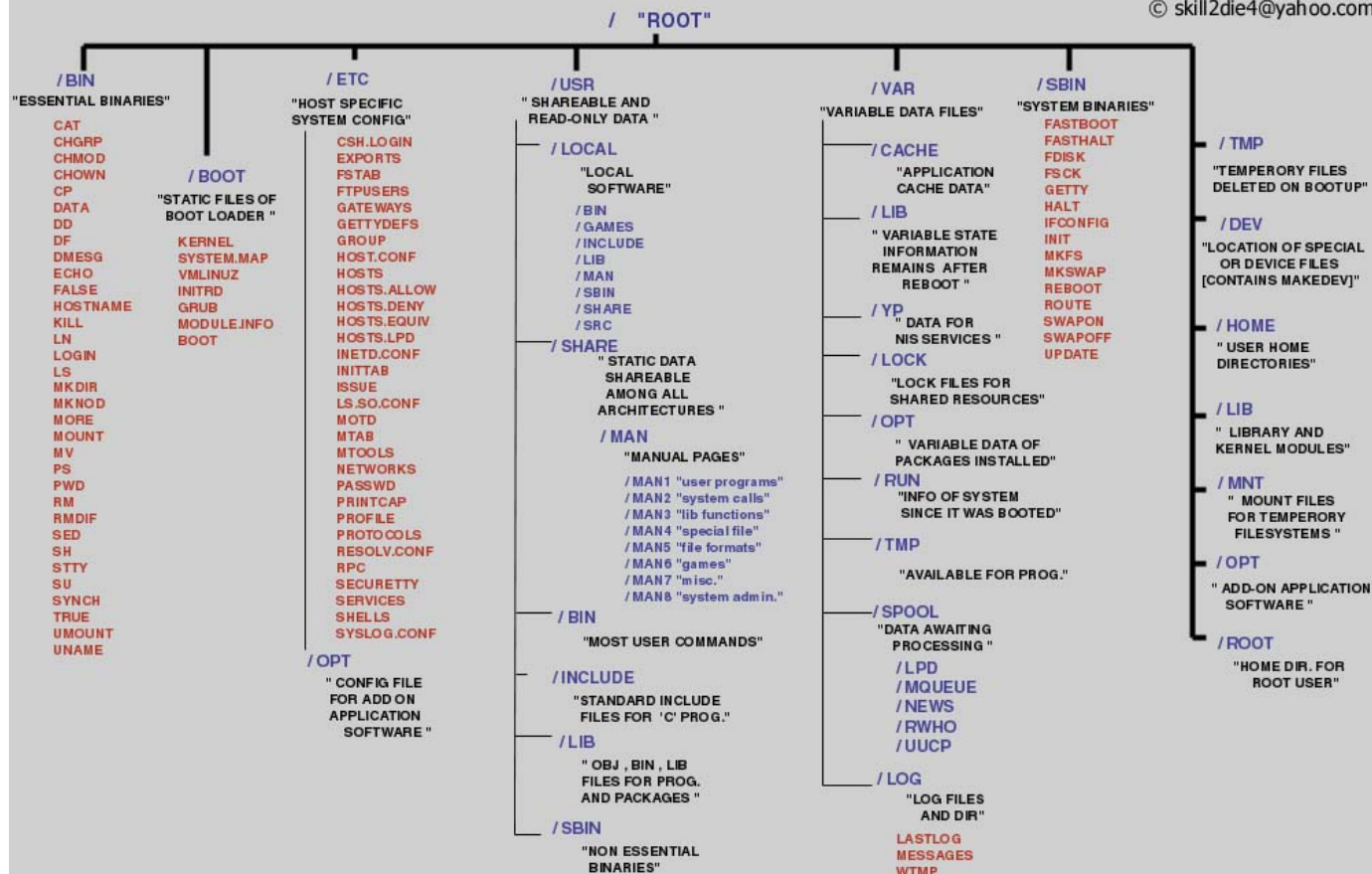


Estructura del sistema de ficheros o archivos. Autor: Miguel Ángel Sarmiento: https://youtu.be/E_IHBKMrfOU

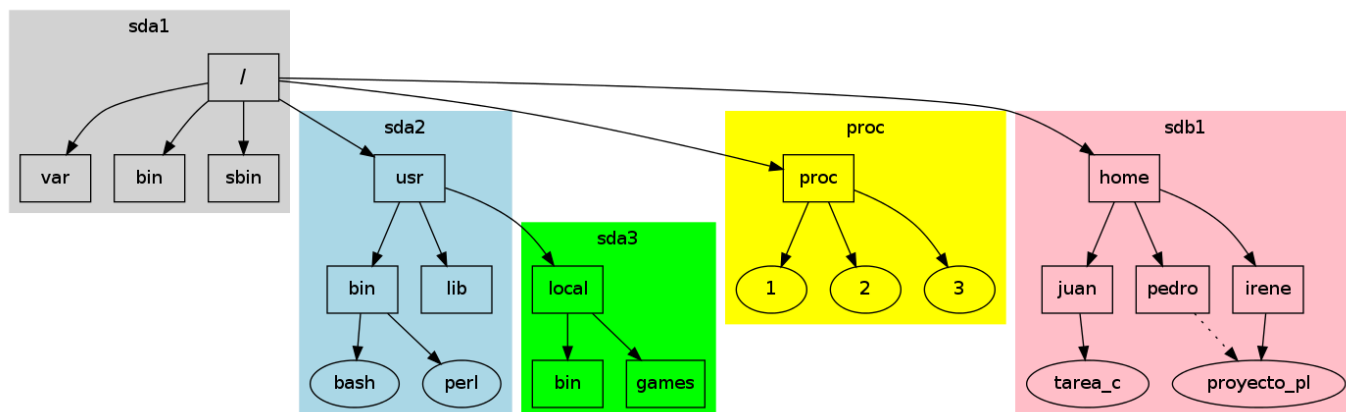


Para saber más

En la siguiente imagen -que se incluye a modo orientativo- se puede apreciar un mayor detalle en el contenido y finalidad de los diferentes directorios del sistema. Mención especial a varios de los ficheros/comandos que iremos trabajando a lo largo del curso: cat, chmod, chown, mount, ..., fstab, hosts, passwd, ..., fdisk, ifconfig, halt, reboot, etc.



En la imagen siguiente puedes ver cómo en Linux cada uno de los directorios y subdirectorios, aunque siempre será accesible desde / puede que esté montado desde una partición o dispositivo diferente:

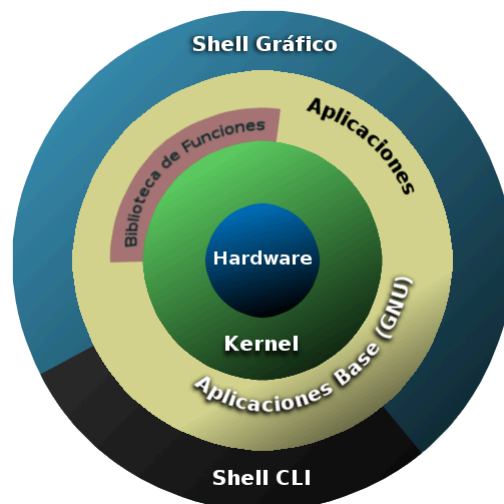


2. La shell bash

Para manejar GNU/Linux se puede hacer de dos formas: una gráfica y una texto conocida como consola o terminal.

Linux ofrece el mecanismo de consolas o terminales virtuales. Este consiste en que a partir de una **entrada** (el **teclado**) y con una **salida** (el **monitor**) se simulen varias terminales, donde el mismo, o distintos usuarios puedan conectarse indistintamente. De esta forma es posible tener más de una sesión abierta en la misma máquina y trabajar en ellas.

Por defecto, en las versiones más recientes de Ubuntu, las **consolas** desde la tres a la seis (**tty3**, ..., **tty6**) tienen asociado un



programa que permite conectarse al sistema en **modo texto**, mientras que la uno y dos (**tty1 y tty2**), si se instaló y activó el "modo gráfico", constituyen la **consola gráfica**.

Q Wikipedia.

Arquitectura de Linux por capas

El cambio de una consola a otra se realiza a través de la combinación de teclas Alt y Fx (las teclas de Función), donde x oscila entre 1 y 6 (aunque puede haber más). Desde una consola gráfica para cambiar a otra tipo texto se debe además presionar la tecla Ctrl, pues las combinaciones Alt + Fx son capturadas e interpretadas por las aplicaciones gráficas de otra forma.

Si nos remitimos a una consola texto podremos apreciar que en ella se mostrará, el nombre que se le asignó al sistema en la instalación y la palabra login. Aquí puede entrarse el login o nombre identificador de un usuario del sistema. Luego se pedirá **el password que al ser introducido no se mostrará ningún eco o cursor en la pantalla (para que nadie sepa ni el nº de caracteres que lo componen)**. Si ambos son válidos se establecerá la conexión y se mostrará el prompt:

usuario@maquina:~\$

Del prompt podemos obtener la siguiente información:

usuario: El usuario que está manejando la Shell.

maquina: El nombre del pc que se está manejando.

~: Directorio, del sistema de archivos donde se encuentra el usuario. El caracter ~ **virgüllla** simboliza el directorio personal del usuario que en ese momento está manejando el Shell.

\$ ó #: Indica si es un usuario normal (\$), o es un usuario con privilegios, normalmente root (#).

De esta manera, con lo que se acaba de explicar: **usuario@maquina:~\$** sería lo mismo que **usuario@maquina:/home/usuario\$** y **root@maquina:~#** sería equivalente a **root@maquina:/root#**

El programa que se ejecuta en Linux cada vez que un usuario se conecta y que le permite interactuar con el sistema se conoce como shell (a partir de este punto cuando nos refiramos a "**shell**" nos estaremos refiriendo siempre a la shell modo texto o línea de comandos: **Shell CLI** en la imagen). Éste es capaz de interpretar una gran gama de comandos y sentencias y por tanto constituye a su vez un poderoso lenguaje de Scripting o programación.

Existen muchos tipos de shells orientados a caracteres para Linux. Ejemplos:

- ❑ **Bourne shell (sh)** -- Escrita por **Steve Bourne**, cuando estaba en **Bell Labs**. Se distribuyó por primera vez con la **Version 7 Unix**, en **1978**, y se mejoró con los años.
- ❑ **Bourne-Again shell (bash)** -- Se escribió como parte del proyecto **GNU** para proveerlo de un superconjunto de funcionalidad con respecto a la shell Bourne.
- ❑ **Z shell (zsh)** -- Considerada como la más completa: es lo más cercano que existe en abarcar un superconjunto de sh, ash, bash, csh, ksh, y tcsh.

De los mencionados anteriormente **bash es el más empleado en Linux**. Es uno de los más desarrollados y está portado para múltiples plataformas. Ofrece entre otras posibilidades las siguientes:

- ❑ **Auto completar durante la escritura.** Al teclear uno o varios caracteres se puede pulsar TAB con el objetivo de que en caso de que pueda completarse de forma unívoca un comando, nombre de fichero o una variable (en dependencia del contexto), complete de forma automática (se escriba el resto de la palabra). Si existieran varias posibilidades para completar la palabra, se oirá un sonido y volviendo a pulsar TAB se mostrarán en pantalla todas las posibilidades existentes.
- ❑ **Historial de comandos.** Esta es una facilidad de muchos otros shells que permite el movimiento a través de los últimos N comandos ejecutados, en la sesión actual o en las anteriores. N por defecto es 1000, pero puede modificarse. Para moverse arriba y abajo se suelen utilizar los cursores.

- **Poderosas estructuras de control para realizar `scripts`** también llamados Procesos por lotes. Se pueden usar instrucciones `if`, `for`, `while`, `select`, etc.
- **Definición de funciones y alias para comandos.** Las funciones permiten definir subrutinas programadas usando el lenguaje de `bash` y los alias, asociar nombres a invocaciones a comandos con ciertas opciones y argumentos de forma más nemotécnica o abreviada.



La terminal, primera toma de contacto

Para que vayas comprobando cómo funciona una terminal sigue y vete practicando con este vídeo:

Ubuntu 8.- La terminal (shell bash) Primera ...  Q La terminal (shell bash) Primera toma de contacto: https://youtu.be/VFv3mrWeSdU	
--	---

Formas generales de obtener ayuda en Linux

Existen múltiples y variadas formas de obtener ayuda en un sistema Linux:

- Muchos comandos poseen una opción para mostrar una ayuda breve acerca de su utilización. Esta opción usualmente es `-h`, `--help` ó `-?`.
- El comando **man** formatea y despliega un manual bastante amplio acerca de comandos, formatos de ficheros de configuración, etc. Los manuales están disponibles y pueden instalarse en múltiples idiomas.

Ejemplo: `$ man chmod`

Una vez en la página del manual de `man`, es importante que recuerdes que:

Flechas arriba/abajo: retrocede/avanza línea/palabra <Enter>: busca palabra

n: busca siguiente palabra (**n**ext=próximo)

p: busca palabra previa (**p**revious=previo)

- Existe un conjunto de comandos integrados (builtin) al bash que no poseen un manual propio. Para ellos se puede emplear el comando **help**. Algunos de los comandos integrados al bash son: cd, fg, bg, logout, exit, umask, set, help, source, alias, echo, kill, jobs y export.

Ejemplo: **\$ help logout**

- Los comandos whereis y apropos:

para saber dónde está el programa, el fuente y las páginas de ayuda del comando mkdir

\$ whereis mkdir

para saber los programas/comandos que contienen en su nombre y descripción la palabra "docker"

\$ apropos docker

- Muchos programas se empaquetan con su documentación en diversos formatos. Normalmente estas ayudas se agrupan en el directorio **/usr/share/doc**

Rutas absolutas y relativas

Para indicar la ubicación en disco de un fichero hay que indicar la lista de directorios que contienen al fichero. Es decir, un fichero puede estar dentro de un directorio que a su vez está dentro de otro y así varios. En esta lista, que denominaremos ruta de acceso, cada directorio está separado del siguiente directorio por el signo / , y sin dejar espacios en blanco.

Para facilitar las cosas existen ciertos directorios especiales:

- / Es el directorio raíz. El superior de todos.
- . Es el directorio activo. En el que nos encontramos en un momento dado.
- .. Es el directorio superior al que nos encontramos.

Hay dos formas de expresar la ruta de un fichero, absoluta y relativa.

Una **ruta absoluta** es aquella que **parte del directorio raíz**. Las rutas absolutas son válidas en cualquier caso. Si establecemos como ejemplo paralelo la dirección de la vivienda, la dirección absoluta incluye la calle y el número de la vivienda, de forma única e independiente.

Una **ruta relativa**, es una ruta que **parte del directorio actual** como origen. Esta ruta sólo es válida desde un directorio actual concreto, es decir es relativa a un directorio. Si seguimos el ejemplo que pusimos para la dirección de una vivienda, una ruta relativa es como si te indicaran "dos casas más allá de cierta esquina". Observamos como la dirección "dos casas más allá una cierta esquina" es sólo válida para una calle, y esa misma dirección referida a otra calle sería otro inmueble distinto.

Ejemplo:

El archivo fich.txt se encuentra en /home/user/docu/text

Si estás en /home/user/ la **ruta absoluta** hacia fich.txt es

/home/user/docu/text/fich.txt

Si estás en /home/user/ la ruta relativa hacia fich.txt es

docu/text/fich.txt

Si estás en /home/user/Escritorio, la ruta relativa hacia fich.txt es

2.1. Comandos básicos

Ubuntu | 12.- La terminal (shell bash) | 1.- Como...



La terminal (shell bash) | 1.- Comodines: <https://youtu.be/YQoDV8S3BrI>



En resumen, en los comandos bash, vamos a poder usar expresiones regulares con metacaracteres:

* : sustituye a cualquier carácter o conjunto de caracteres.

? : sustituye a un solo carácter.

[] : se especifica que carácter o caracteres se sustituirán en esa posición.

En las siguientes secciones mencionaremos y describiremos brevemente la utilidad de un grupo de comandos básicos para interactuar con un sistema Linux. La recomendación es que los vayas probando según los vas estudiando con los ejemplos propuestos u otros que se te ocurran. Para su mejor comprensión clasificamos los comandos en varias categorías:

Comandos para manipular ficheros y directorios

El **comando ls** permite listar el contenido de un directorio. Sintaxis: `ls [opciones] [directorio | fichero]`

Algunas opciones:

-l : muestra la salida en formato largo.

-R: lista recursivamente un directorio.

-a : lista además los ficheros ocultos (en linux los ficheros ocultos son los que sus nombres comienzan con punto).

-h : muestra el tamaño de los ficheros en forma más legible, para "h"umanos (Ej.: 16M, 4k,)

Ejemplos:

\$ ls -lh /etc

\$ ls -R /usr

\$ ls -la

Un grupo de opciones que se suele utilizar bastante es "la" (ls -la)

Ubuntu | 8.- La terminal (shell bash) | Primera to...



Q

*La terminal (shell bash) | Primera toma de
contacto: <https://youtu.be/VFv3mrWeSdU>*



El **comando cd** se utiliza para cambiar el directorio actual.

Sintaxis: cd [directorio]

Ejemplos:

\$ cd /tmp

cambia hacia el directorio base del usuario actual

\$ cd

cambia al directorio home del usuario actual

\$ cd ~

el caracter ~ se puede insertar generalmente en los teclados españoles con la combinación AltGr+4 (tecla 4 la que está encima de las teclas ER) y barra espaciadora o también con Alt+126 (teclea el número 126 desde el teclado numérico mientras se mantiene pulsada la tecla Alt)

El **comando pwd** indica el camino absoluto del directorio en el cual nos encontramos actualmente.

Ejemplo: \$ pwd # (nos devuelve algo como /home/alumno/Documentos)

El **comando mkdir** se utiliza para crear directorios. Sintaxis: mkdir [opciones] [directorio]

Algunas opciones:

-p : permite crear una estructura completa de directorios y subdirectorios

Ejemplos:

```
$ mkdir datos
```

```
$ mkdir /bin
```

con el siguiente comando

```
$ mkdir -p docs/linuxdocs/howtos/pdf
```

se crean los directorios intermedios si es necesario, para entenderlo mejor, después de ejecutarlo ejecuta:

```
$ ls -Rl docs
```

El **comando touch** se utiliza para cambiar la fecha de acceso y/o modificación a un archivo; si el argumento archivo corresponde al nombre de un archivo que no existe, touch creará el archivo con dicho nombre y sin ningún contenido. Sintaxis: touch [opciones] archivo

Ejemplos:

```
$ touch fich1
```

Ubuntu | 12.- La terminal (shell bash) | 2.- Manejo ...



*La terminal (shell bash) | 2.- Manejo de ficheros y directorios
(I): <https://youtu.be/CsbG2mOdkvg>*



El **comando rmdir** se utiliza para borrar directorios vacíos

Ejemplo: \$ rmdir datos

El **comando rm** se utiliza para borrar ficheros o directorios

Sintaxis: `rm [opciones] <ficheros | directorios>`

Algunas opciones:

-r: borra recursivamente un directorio (generalmente es más útil que rmdir dado que normalmente no se borran directorios vacíos)

Ejemplos:

```
$ rm prueba
```

```
$ rm -i bin/*
```

```
$ rm -rf temp/
```

El **comando mv** permite renombrar ficheros o directorios. También mueve uno o varios ficheros hacia un directorio.

Sintaxis:

```
mv [opciones] <fuente> <destino>
```

```
mv [opciones] <ficheros> <directorio>
```

Ejemplos:

```
# renombra un fichero
```

```
$ mv mail.cf mail.cf.old
```

```
# mueve ficheros terminados en .txt al directorio /tmp
```

```
$ mv *.txt /tmp
```

El **comando cp** permite copiar un fichero en otro, o varios ficheros en un directorio. Sintaxis:

```
cp [opciones] <fuente> <destino>
```

```
cp [opciones] <ficheros> <directorio>
```

Algunas opciones:

-r: copia recursivamente un directorio.

Ejemplos:

```
# copia un fichero en el directorio actual
```

```
$ cp /etc/passwd .
```

```
#copia los ficheros terminados en sh en tmp
```

```
$ cp /usr/bin/*sh /tmp
```

Ubuntu | 12.- La terminal (shell bash) | 2.- Manejo d...



*La terminal (shell bash) | 2.- Manejo de ficheros y directorios
(II): <https://youtu.be/EWYWxu9FJnQ>*



Comandos para paginar, visualizar y editar ficheros

El **comando cat** concatena (conCATenate) ficheros y los imprime en la salida estándar. Si solo se da el origen a cat, utiliza como salida la pantalla. Es decir, cat carta.txt muestra por pantalla el fichero carta.txt. Si solo se da la salida a cat (cat > fichero) utiliza como entrada el teclado.

Ejemplo:

```
# cat /etc/passwd /etc/shadow
```

```
$ cat > fichero
```

El comando **less** pagina (dividen en páginas) uno o varios ficheros y los muestran en la terminal (pantalla). A diferencia de otros comandos less acepta el empleo de las teclas de flecha arriba/abajo ↑ ↓ para moverse por el texto.

Ejemplos:

```
$ less /etc/passwd
```

```
$ cat fichero | less
```

Algunas teclas que podemos usar mientras usamos estos programas son:

q - permite interrumpir el proceso y salir (quit).

/patrón - realiza búsquedas del "patrón" dentro del texto. Para repetir la búsqueda del mismo patrón sólo es necesario escribir n.

El comando **tail** se utiliza para mostrar las últimas líneas de un archivo de texto. Es especialmente útil para monitorear archivos de registro (logs) en tiempo real con la opción -f o para revisar rápidamente el contenido reciente de un archivo grande sin abrirlo por completo:

```
tail -f /var/log/syslog
```

```
tail -n 20 /var/log/syslog # para ver las últimas 20 líneas del fichero (por defecto son 10)
```

Ubuntu | 12.- La terminal (shell bash) | 4.- Edición, mu...



*La terminal (shell bash) | 4.- Edición, muestra y filtrado de ficheros de texto
(!): <https://youtu.be/1FyseHxmch4>*



Comandos para hacer búsquedas de patrones en el contenido de ficheros

El **comando grep** busca patrones **en el contenido de ficheros**. Por defecto devuelve todas las líneas que contienen un patrón (cadena de texto) determinado en uno o varios ficheros. Utilizando las opciones se puede variar mucho este comportamiento.

Sintaxis: `grep [opciones] <patrón> [ficheros]`

Algunas opciones:

-r busca en un directorio de forma **recursiva** (subdirectorios incluidos)

-n imprime el número de cada línea que contiene al patrón.

Ejemplos:

```
$ grep linux /usr/share/doc
```

```
$ grep root /etc/passwd
```

```
# grep -n error /var/log/syslog
```

Ubuntu | 12.- La terminal (shell bash) | 4.- Edición, mu...



*La terminal (shell bash) | 4.- Edición, muestra y filtrado de ficheros de texto
(II): https://youtu.be/P3v1_VOBUqE*



Otros Comandos

El comando **echo** muestra en su salida todo lo que se le pase como argumento. Ejemplo:

```
$ echo Hola amigo...
```

Hola amigo...

El comando **clear** limpia la pantalla del terminal Ejemplo:

```
$ clear
```

El comando **alias** permite asignarle otros nombres a los comandos. Sintaxis: `alias [nombre[=valor]...]`

Sin argumentos muestra todos los alias definidos por el usuario actual. Para deshabilitar un alias se emplea el comando `unalias`.

Ejemplos:

```
$ alias ll='ls -l --color'
```

El comando **df** permite saber el espacio libre en los dispositivos montados Sintaxis: `df [opciones] [directorios]`

Algunas opciones:

`-h` : imprime de forma más representativa (Ej. M para megabytes).

Ejemplos:

```
$ df -h /
```

```
$ lsb_release -a #muestra toda la información sobre la distro en uso
```

El comando **exit** permite terminar el shell actual. Si se tiene un único shell es equivalente a desconectarse del sistema, pero si se está en un subshell (también llamada shell hijo) sólo se terminará este, retornando al shell anterior.

2.2. Manejo avanzado de la Shell

Variables de entorno

Las variables de entorno son un conjunto de valores dinámicos que normalmente afectan el comportamiento de los procesos en una computadora. Existen variables definidas en el sistema, y pueden verse con el comando **env**.

Existen dos variables muy comunes en Linux:

- **\$PATH** Contiene una lista separada por dos puntos de directorios en los cuales el intérprete de comandos buscará los archivos ejecutables que no se invocan con una ruta. Para ver su contenido ejecutar

```
echo $PATH
```

- **\$HOME** Contiene la ubicación del directorio de usuario. De esta manera, los dos comandos siguientes tienen el mismo efecto:

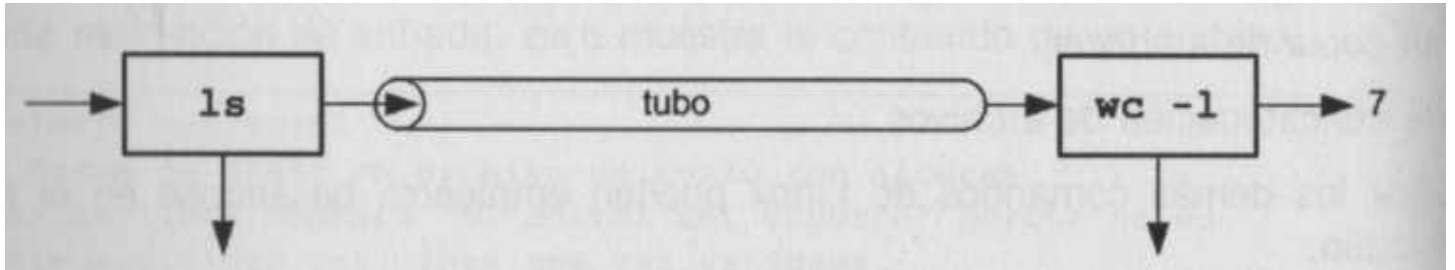
```
cd ~
```

```
cd $HOME
```

La forma de crear una variable y asignarle valor es: **NOMBRE_VARIABLE=valor**

Tuberías

Las tuberías en GNU/Linux enlazan la salida de un comando con la entrada de otro, lo que nos permite realizar tareas complejas en la línea de comandos. Una tubería se escribe con el carácter **|** (atajo [AltGr+1]).



Veamos un ejemplo: **ls | wc -l**

El comando **wc** cuenta el número de líneas, palabras y caracteres de un archivo. La línea anterior nos muestra, por tanto, el número de líneas del listado del contenido del directorio actual en el que nos encontremos.

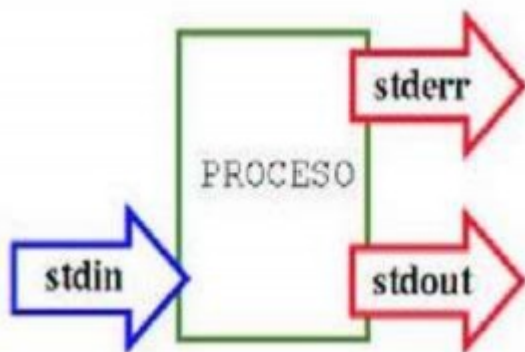
Ubuntu | 12.- La terminal (shell bash) | 7.- Concat...



*La terminal (shell bash) | 7.- Concatenación de órdenes
(l): https://youtu.be/TYV2VLHfx_Y*

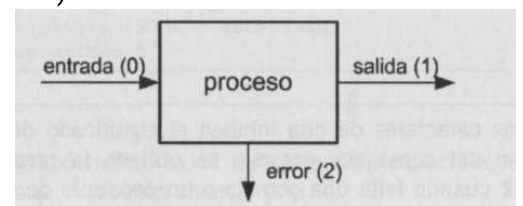


Redireccionamientos



Los comandos toman su entrada de algo conocido como entrada estándar y envían su salida a la salida estándar (a menudo abreviado como "stdin o standard input" y "stdout o standard output"). Además existe una salida especial para los mensajes de error de cada programa (stderr o standard error).

Se llaman "entradas" a los parámetros



tratados por el proceso, y "salidas" a los resultados obtenidos. Los "errores" corresponden a un resultado proveniente de un error de tratamiento por parte del proceso. Los tres descriptores de archivo correspondientes también se identifican por un número: 0 para entrada, 1 para salida y 2 para error.

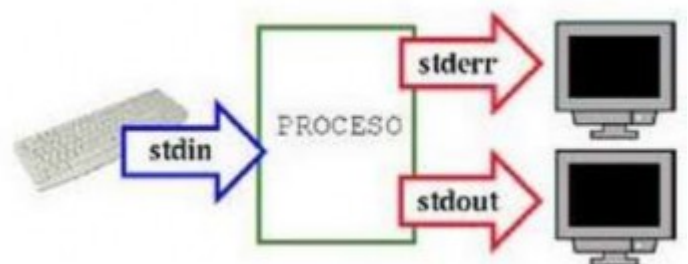
Como puede verse en la gráfica, el intérprete de comandos configura el sistema **por defecto** de forma que la **entrada estándar es el teclado** y la **salida/errores la pantalla**.

Podemos cambiar este funcionamiento, por ejemplo:

Redirigir salida/stdout en vez de por pantalla a un archivo **creando** el archivo:

> archivo #también podría haberse puesto 1>archivo

Redirigir **stdout** al archivo, pero **si** el fichero **existe añade** el texto:



>> archivo #también podría haberse puesto 1>>archivo

Redirige **stderr** al archivo

2> archivo

Redirige **stdin** desde el archivo

< archivo

Ejemplos:

ls /etc > fich.txt ; el resultado del listado lo meterá en un fichero

sudo apt update 2>errores.txt # si hay error en algún repositorio el resultado lo manda al fichero errores.txt

\$ cat /etc/passwd | grep juan

\$ ls -lia | grep "carta"

date >> fechas.txt # Añade la fecha al fichero. Podemos hacerlo varias veces.

wc < /etc/passwd # cuenta las líneas, palabras y caracteres del fichero passwd con los usuarios del sistema

Ubuntu | 12.- La terminal (shell bash) | 5.- Redirec...



*La terminal (shell bash) | 5.-
Redireccionamientos: <https://youtu.be/7ypnZDVcvxE>*



2.3. Ejercicios opcionales



Ejercicios para practicar :: Apartado Comandos básicos

El objetivo de estos ejercicios es que practiques todos los comandos explicados para que los comprendas e integres. Son muy básicos por lo que si tienes alguna duda sobre ellos plantéala a tu

tutor en los mecanismos de comunicación habituales. Este tipo de ejercicios son fáciles de copiar por lo que deberás entregar capturas de pantalla en las que se pueda apreciar:

- el comando que se ejecuta
- los mensajes y el resultado de su ejecución
- fecha y hora (en Ubuntu suele salir arriba a la derecha en un reloj configurable para que se vea también "Fecha y mes")

Podremos usar la consola del entorno gráfico o abrir una consola texto pulsando Ctrl + Alt + F2.

Parte 1

1. Crear una estructura de directorios (dir) y ficheros (f) en vuestro home que incluya:

- ej1
 - dir1
 - dir11
 - f3
 - dir2
 - f4
 - f1
 - f2

2. Mover el fichero f3 al directorio ej1
3. Copiar el fichero f4 en el directorio dir11, suponiendo que el directorio actual es dir2. Hazlo otra vez estando en dir1. Usa rutas relativas.
4. Crear el directorio copia dentro del ej1 y copiar en él todo el contenido de dir1.
5. Renombrar el fichero f1 por el nombre docu y ver los permisos de ese fichero
6. Borrar toda la estructura creada

Parte 2

7. Crea un directorio llamado ej2 en tu home
8. Dentro del directorio ej2 crea otros dos directorios llamados exa y respu.
9. Desde el directorio en el que te encuentras lista el contenido del directorio /etc de forma recursiva y con salida en formato largo.
10. Crea con nano un fichero llamado examen1.txt en el directorio de exa y escribe en él dos preguntas.
11. Crea con nano un fichero llamado respuestas.txt en el directorio de exa y escribe en él dos respuestas.
12. Cambia el nombre del fichero respuestas.txt por el de respuestas1.txt
13. Mueve todos los ficheros .txt del directorio de exa al de respu.
14. Desde el directorio exa crea un directorio recupera en tu home
15. Busca la ayuda del comando ls
16. Muestra el contenido del fichero examen1 por páginas.
17. Busca en tu directorio personal todos los archivos con extensión .txt y cuyo nombre comience por r.

Parte 3

18. Crear una carpeta ej3 en tu home
19. Crear un fichero lista.txt que contenga el listado de todos los archivos de tu directorio home de forma recursiva, incluyendo los archivos ocultos
20. Crear un nuevo fichero con las 3 últimas líneas del fichero lista.txt
21. Busca todos los archivos del disco duro que comiencen por net y guarda el listado de todos ellos en un archivo que se encuentre en el directorio ej3 llamado net.txt
22. Situados de nuevo en la primea consola añade al fichero net.txt información sobre el espacio ocupado y libre del disco. Dichos datos deben ser entendibles por el usuario (MB, GB...).

Parte 4

23. Crea un alias para que al poner la palabra `direction` me realice la búsqueda del punto 21.
 30. Crear los directorios `dir1`, `dir2` dentro de `ej3`. Dentro de `dir1` crea el directorio `dir11`. Dentro del directorio `dir2` crea el directorio `dir21`.
 31. Copia al directorio `dir21` los ficheros del directorio `/dev` que empiecen por `t`, acaben en una letra que vaya de la `a` a la `b` y tengan 5 letras en su nombre.
 32. Borra los archivos del directorio `dir21` que no acaben en `b` y tengan una `q` como cuarta letra.
-



Ejercicios para practicar :: Apartado Manejo avanzado de la Shell

1. Crear un nuevo fichero de nombre `líneas.txt` con las 5 últimas líneas del fichero `/etc/passwd`
 2. Añade al `líneas.txt` las 5 primeras líneas de `/etc/passwd`
 3. Muestra el contenido del archivo por pantalla para asegurarte que se ha creado correctamente.
 4. Añade al fichero `líneas.txt` el/los nombre/s de/los fichero/s de tu directorio `home`, que contengan en su nombre la cadena `"ai"`.
 7. Busca todos los ficheros en `/etc` cuyo nombre empiece por `"de"` y guarda dicho listado, en un archivo llamado `ficherosde` de tu directorio de trabajo.
 8. Crea un alias del comando `pwd` para que podamos comprobar donde estamos con el comando `"donde_estamos"`.
-

3. Gestión de usuarios y grupos

Tipos de Usuarios

Linux es un sistema multiusuario, permite que varios usuarios usen el sistema simultáneamente. Cada usuario tendrá sus permisos y podrá o no realizar ciertas tareas, además un usuario podrá manejar sus documentos con total independencia de los demás.

Linux establece **login y password** (nombre y contraseña) para cada usuario. Las contraseñas son almacenadas de forma cifrada. Todo usuario pertenece al menos a un grupo. Un grupo permite administrar permisos a un conjunto de usuarios, evitando tener que dárselos a cada usuario individualmente

Los usuarios en Unix/Linux se identifican por un número único de usuario **User ID, UID**. Y pertenecen a un grupo principal de usuario, identificado también por un número único de grupo **Group ID, GID**. El usuario puede pertenecer a más grupos además del principal.

Usuario root

También llamado superusuario o administrador, tiene privilegios para realizar todo tipo de tareas administrativas.

- ☐ Su UID es 0
- ☐ Es la única cuenta de usuario con privilegios sobre todo el sistema. Controla la administración de cuentas de usuarios (por ello a partir de ahora en muchos de los ejemplos veremos `#` en lugar de `$` delante de los comandos de ejemplo, esto indica que los comandos se ejecutan como root por lo que no hará falta poner `sudo`).
- ☐ Ejecuta tareas de mantenimiento del sistema.

- ❑ Instala software en el sistema.

Usuarios especiales

Existen otros usuarios especiales, que sistema operativo usa para determinadas operaciones. Estos usuarios del sistema no pueden entrar al sistema.

Ejemplos: bin, daemon, sync, mail, operator, squid, apache, etc.

- ❑ Generalmente se les asigna un UID entre 1 y 100
- ❑ No tiene todos los privilegios del usuario root, pero asumen algunos de sus privilegios.
- ❑ Están creados para hacer el sistema más seguro.
- ❑ No tienen contraseñas pues son cuentas que no están diseñadas para iniciar sesiones con ellas.
- ❑ Se crean automáticamente al momento de la instalación de Linux o de un servicio.

Usuarios Normales

Son los usuarios que conocemos habitualmente, los creamos para poder entrar al sistema operativo.

- ❑ Normalmente tienen UID superior a 1000
- ❑ Cada usuario dispone de un directorio de trabajo único, ubicado en /home.
- ❑ Sólo tienen privilegios totales en su HOME
- ❑ Cada usuario puede personalizar su entorno de trabajo.

Ficheros de Usuarios en el sistema

Toda la información sobre usuarios, grupos y contraseñas se guarda en los archivos:

- ❑ **/etc/passwd**: información (nombre, home, Shell, etc.) del usuario.
- ❑ **/etc/group**: información sobre grupos
- ❑ **/etc/shadow**: contraseñas cifradas de los usuarios

En estos archivos de texto, se almacena la información línea a línea (cada una es un usuario o un grupo). Dentro de cada línea hay varios campos separados por ":".

/etc/passwd

Cualquiera que sea el tipo de usuario, todas las cuentas se encuentran definidas en este archivo de configuración.

El archivo /etc/passwd contiene una línea para cada usuario:

```
root:x:0:0:root:/root:/bin/bash
...
saned:x:114:123::/home/saned:/bin/false
delarioja:x:1000:1000:delarioja,,,:/home/delarioja:/bin/bash
neo:x:1001:1001:,,,:/home/neo:/bin/bash
trinity:x:1002:1004:,,,:/home/trinity:/bin/bash
```

usuario : contraseña : UID : GID : comentarios : directorio_home : shell

- ❑ contraseña: aparece una x que indica que la contraseña está cifrada en /etc/shadow
- ❑ directorio_home: directorio de inicio del usuario
- ❑ shell: intérprete de comandos del usuario; si indica /bin/false no se podrá entrar al sistema -suele tratarse de cuentas de usuario creadas con otros propósitos como la ejecución de servicios/programas-

/etc/group

Este archivo guarda la relación de los grupos a los que pertenecen los usuarios del sistema, contiene una línea para cada usuario con tres o cuatro campos por usuario:

```
root:x:0:root
...
delarioja:x:1000:
sambashare:x:124:delarioja
neo:x:1001:
sion:x:1002:neo,trinity
nabuco:x:1003:neo,trinity
trinity:x:1004:
```

grupo : x : GID : lista_usuarios

Donde grupo indica el nombre del grupo creado, x indica que el password se guarda en el archivo shadow, GID es el identificador del grupo y la lista de usuarios es como su nombre lo indica todos los usuarios que pertenecen a ese grupo.

/etc/shadow

Anteriormente (en sistemas Unix) las contraseñas cifradas se almacenaban en el mismo /etc/passwd. El problema es que „passwd” es un archivo que puede ser leído por cualquier usuario del sistema, aunque solo puede ser modificado por root. Con cualquier computadora potente de hoy en día, un buen programa de descifrado de contraseñas y paciencia es posible “crackear” contraseñas débiles. El archivo shadow, resuelve el problema ya que solo puede ser leído por root. Considérese a shadow como una extensión de passwd ya que no solo almacena la contraseña encriptada, sino que tiene otros campos de control de contraseñas.

Cada una de las líneas representa un usuario y responde al esquema:

```
saned*:15937:0:99999:7:::
delarioja:$6$vakm6Gc1$eV8kbt/bB0cfA.PiI9bPucG3Fk3VEuN5l0gGUv4GTG73eLAF5EhbTnp6k/Hng4DY6JKc
neo:$6$YCHrvRvc$LiwJeiAnnXpmWTEVw7Bxq8oThMocwtY2a0mkvrMNAsFBr4B5eMtTKo6m8O6fgVh0ajUv
trinity:$6$9SGD3syx$j3jgwzLe1Z2hNmT6EhhSgXis.a32LsVHnHU5DuWyztXvB5H.laHO96L1eqXOJOfen2hOo
```

en el caso de la línea del usuario root en Ubuntu, si no se ha activado todavía su contraseña tal y como indicamos en el apartado relativo al root de este tema, aparecerá una exclamación ! indicando que no está activado y por tanto que no puede iniciar sesión:

```
root!:15975:0:99999:7:::
```

si ya se le dio contraseña con el comando sudo passwd root la apariencia de la línea será similar a esta:

```
root:$6$lnDqRI/F$WMg5y/54LMKgt8VKELm1lnUXeWf9FfeGv3tko39kYaUWirAU9wK.bB9crv.LK8VGT0h6SVv
```

Comandos para manipular usuarios y grupos

En linux tenemos 3 formas de agregar usuarios, una mediante el comando adduser, otra mediante el comando useradd y la última usando alguna herramienta con interfaz gráfica que tenga instalado nuestro sistema.

La diferencia entre los comandos useradd y adduser es que adduser agrega el usuario de manera interactiva y es un enlace a useradd, si usamos adduser el comando nos solicitará los datos que sean necesarios para agregar el nuevo usuario y si ejecutamos useradd tendremos que hacer algunas de las tareas de agregar un usuario de manera manual o pasándole parámetros directamente.

adduser

El comando adduser permite añadir nuevos usuarios al sistema y les asigna una información por defecto. Se encuentra enlazado simbólicamente por el nombre useradd que es el original de UNIX, pero con adduser

(que también se puede usar) es más fácil ya que automatiza ciertos parámetros (proceso que se describe más abajo).

Sintaxis: `adduser [opciones] <usuario>` Ejemplos:

crea el usuario morfeo con las propiedades por defecto

```
# adduser morfeo
```

```
# useradd -d /home/morfeo -s /bin/bash morfeo
```

```
# adduser morfeo
```

Añadiendo el usuario `morfeo' ...

Añadiendo el nuevo grupo `morfeo' (1005) ...

Añadiendo el nuevo usuario `morfeo' (1003) con grupo `morfeo' ...

Creando el directorio personal `/home/morfeo' ...

Copiando los ficheros desde `/etc/skel' ...

Introduzca la nueva contraseña de UNIX:

Vuelva a escribir la nueva contraseña de UNIX:

passwd: contraseña actualizada correctamente

Cambiando la información de usuario para morfeo

Introduzca el nuevo valor, o presione INTRO para el predeterminado

Nombre completo []:

Número de habitación []:

Teléfono del trabajo []:

Teléfono de casa []:

Otro []:

¿Es correcta la información? [S/n]

deluser

El comando `deluser` permite eliminar definitivamente un usuario del sistema. También enlaza a `userdel`.

Sintaxis: `deluser [opciones] <usuario>` Opciones:

`--remove-home`: borra su directorio home

Ejemplo: elimina al usuario morfeo y borra su HOME

```
# deluser --remove-home morfeo
```

passwd

El comando `passwd` permite cambiar el password de un usuario. También puede bloquear, desbloquear y deshabilitar una cuenta. Si se invoca sin argumentos se asume el usuario actual.

Sintaxis: `passwd [opciones] <usuario>` Ejemplos:

cambia la password del usuario que lo ejecuta

```
# passwd
```

coloca una contraseña para morfeo

```
# passwd morfeo
```

usermod

El comando `usermod` modifica propiedades de los usuarios como: el login, el directorio personal, el shell que se inicia al conectarse, los grupos a los que pertenece, la fecha de expiración de la cuenta, etc. También bloquea y desbloquea una cuenta.

Sintaxis: `usermod [opciones] <usuario>`

Ejemplos:

establece morfeo como grupo principal del usuario morfeo y lo añade también a los grupos secundarios/suplementarios sudo y matrix

```
root@ubuntu100-SOM:/home/delarioja/Descargas# usermod -g morfeo -G sudo,mat
root@ubuntu100-SOM:/home/delarioja/Descargas# id morfeo
uid=1003(morfeo) gid=1005(morfeo) grupos=1005(morfeo),27(sudo),1007(matrix)
```

addgroup y delgroup

Los comandos addgroup y delgroup añaden y eliminan un grupo en el sistema respectivamente. También existen groupadd y groupdel

Sintaxis:

```
groupadd [opciones] <grupo>
```

```
groupdel <grupo>
```

Ejemplos:

```
# addgroup matrix
# delgroup matrix
```

Para añadir usuarios a un grupo preexistente utilizaremos indistintamente:

```
# addgroup usuario grupo
# adduser usuario grupo
```

su

El comando su permite ejecutar un shell (u otro comando) cambiando los identificadores del grupo y del usuario actual. Si no se especifica el login del usuario se asume root.

Sintaxis: su [opciones] <usuario> Ejemplos:

```
$ su -
# su morfeo
```

sudo

El comando sudo permite a un usuario normal (siempre que pertenezca al **grupo** admin o **sudo en Ubuntu**) ejecutar otros comandos como si fuera el usuario root. Los usuarios y los comandos permitidos para el comando sudo se configuran en el fichero /etc/sudoers

Sintaxis: sudo <comando> Ejemplos:

```
$ sudo adduser morfeo
```

3.1. Permisos de acceso

Permisos

El sistema de permisos en Linux se basa en un esquema de usuarios/grupos que lo convierte en la base principal de la seguridad en Linux, a estos usuarios y grupos se les asignan distintos derechos sobre los archivos y directorios.

Todos los archivos y directorios en Linux tienen permisos que verifican quien puede hacer o no alguna acción con ellos.

La mayoría de los problemas de instalación, configuración y funcionamiento de las aplicaciones en GNU/Linux se debe a derechos de acceso mal adjudicados.

Tipos de Permisos

Los permisos propiamente dichos son tres:

r: read (lectura): Cuando el **permiso de lectura** está activo sobre un **directorio** significa que se podrá **listar los recursos almacenados** en él, si está asignado a un **archivo** se podrá **leer su contenido**.

w: write (escritura): Cuando el **permiso de escritura** está activo sobre un **directorio** significa que se podrá **crear y borrar archivos en su interior**, si esta activado para un **archivo** significa que se podrá **modificar su contenido**.

x: execute (ejecución): Si el **permiso de ejecución** está activo sobre un directorio significa que **el usuario podrá realizar otras funciones dentro de él mediante los otros permisos de lectura y escritura**, y si está activo sobre un **archivo** se podrá **ejecutarlo desde la línea de comandos**.

permiso	archivo	directorio
r	permiso de lectura del contenido del archivo	permiso para listar las entradas del directorio
w	permiso para modificar el contenido del archivo	permiso para modificar las entradas del directorio
x	permiso para ejecutar el archivo	permiso de acceso a las entradas del directorio

¿Cómo verlos?

Para poder ver los permisos de los archivos y directorios es necesario ejecutar el siguiente comando:

```
$ ls -l
```

Este comando nos dará una salida similar a la siguiente:

```
drwxr-xr-x 3 raul raul 4096 2005-02-16 14:47 Desktop
drwxr-xr-x 5 raul raul 4096 2005-02-16 12:42 GNUstep
-rw-r--r-- 1 raul raul 246417 2005-03-03 13:13 foto1.png
-rw-r--r-- 1 raul raul 232505 2005-03-03 13:14 carta2.abw
-rw-r--r-- 1 raul raul 239618 2005-03-03 13:14 informe.abw
drwxr-xr-x 2 raul raul 4096 2005-02-16 12:42 tmp
```

Ahora describamos la salida que hemos obtenido:

El diagrama muestra la siguiente salida de `ls -l` con flechas azules que indican el significado de cada campo:

```
drwxr-xr-x 3 raul raul 4096 2005-02-16 14:47 Desktop
drwxr-xr-x 5 raul raul 4096 2005-02-16 14:47 GNUstep
-rw-r--r-- 1 raul raul 246417 2005-03-03 14:47 foto1.png
-rw-r--r-- 1 raul raul 232505 2005-03-03 16:26 carta2.abw
-rw-r--r-- 1 raul raul 239618 2005-03-03 18:15 informe.abw
drwxr-xr-x 2 raul raul 4096 2005-02-16 14:47 tmp
```

Las flechas azules indican:

- Tipo de Archivo:** Indica el tipo de archivo o directorio (primeros tres caracteres).
- PERMISOS DE PROPIETARIO:** Indica los permisos para el propietario (primeros tres caracteres).
- PERMISOS DE GRUPO:** Indica los permisos para el grupo (caracteres 4-6).
- PERMISOS DE OTROS:** Indica los permisos para otros usuarios (caracteres 7-9).
- Enlaces:** Indica el número de enlaces duros (campo de 10 caracteres).
- Propietario:** Indica el nombre del propietario (campo de 11-12 caracteres).
- Grupo:** Indica el nombre del grupo (campo de 13-14 caracteres).
- Tamaño:** Indica el tamaño del archivo en bytes (campo de 15-16 caracteres).
- Fecha:** Indica la fecha y hora de modificación (campo de 17-18 caracteres).
- Nombre:** Indica el nombre del archivo o directorio (campo de 19-20 caracteres).

```
- rw- r-- r-- 1 raul raul 246417 2005-03-03 13:13 foto1.png
↑ ↑   ↑   ↑   ↑ ↑   ↑   ↑   ↑   ↑   ↑
1 2   3   4   5 6   7   8   9   10  11
```

Con la siguiente línea interpretamos la información así:

1 : Tipo de archivo = es un archivo regular (en contraposición a otro tipo de archivos como p. ej. enlace simbólico llamado comúnmente "acceso directo")

2 : Permisos = los permisos para el **propietario** son de lectura y escritura

3 : Permisos = el **grupo** tiene permiso de sólo lectura

4 : Permisos = los **otros usuarios** tienen el permiso de sólo lectura

5 : Enlace Físico = tiene un enlace duro o hard link (los enlaces duros son diferentes entradas en la tabla de inodos que apuntan al mismo dato)

6 : Propietario = el **usuario raul** es el propietario de este archivo

7 : Grupo = este archivo pertenece al **grupo raul**

8 : Tamaño = su tamaño es de 246417 bytes

9 : Fecha = fue creado el 03 de marzo de 2005

10 : Hora = a las 13:13 horas

11 : Nombre = el archivo se llama foto1.png

Como habrás apreciado los permisos están asignados en grupos de 3 (rwx) y corresponde al:

- **usuario propietario**: dueño del archivo o directorio, en principio es el que lo ha creado
- **grupo propietario**: este grupo es generalmente el grupo principal del propietario del archivo, pero puede ser modificado por éste y tomar el valor de uno de sus grupos secundarios
- **otros**: cualquier usuario diferente del propietario y que no es miembro del grupo propietario

Ejecución de un proceso

Hay que recordar que para que un fichero se pueda ejecutar debe de tener privilegios de ejecución.

Cuando se usa la consola y se escribe el nombre de un ejecutable cualquiera, por ejemplo: nano, lo que se está haciendo es intentando ejecutar un programa que se encuentre en la variable PATH. Es decir el ejecutable tiene que estar en alguno de los directorios que se encuentren en esa variable. En general allí estan, /usr/bin, /usr/local/bin, etc. Por lo tanto al ejecutar el comando "nano" el sistema lo va a buscar en alguno de esos directorios.

Si el programa a ejecutar no está en ninguno de ellos, no funciona. Para ejecutar un programa que se encuentra en un directorio específico simplemente hay que escribir el nombre con su ruta completa (desde cualquier directorio), por ejemplo: /home/usuario/miprograma

También existen dos formas:

1. Situándose donde se encuentra el fichero y usando: ./
2. Poniendo delante la **shell** con la que vas a ejecutar.

Ejemplo:

\$./miprograma.sh # Ejecuta el fichero miprograma.sh

\$ sh miprograma.sh # Ejecuta el fichero miprograma.sh con el intérprete /bin/sh

\$ bash miprograma.sh # Ejecuta el fichero miprograma.sh con el intérprete /bin/bash

Comandos de Permisos

chmod

El comando chmod (**change mode**: cambiar "modo", los derechos de acceso a un archivo se llaman también "modo" en Unix) se utiliza para cambiar los permisos de un fichero. Los permisos se pueden dar en forma de números o con caracteres.

Sintaxis:

chmod [opciones] <ficheros>

Opción: -R en los directorios cambia los permisos recursivamente, es decir incluídos sus ficheros y subdirectorios.

Ejemplos:

```
# chmod 777 foto1.png
# chmod u+w foto2.png
```

chown

El comando chown (**change owner**: cambiar propietario en inglés) se utiliza para cambiar el dueño de un fichero. El dueño de un fichero solo lo puede cambiar el usuario root.

Sintaxis:

chown [opciones] <dueño><ficheros>

Opción: -R en los directorios cambia el dueño recursivamente.

Ejemplos:

```
# chown morfeo foto1.png
# chown -R root datos/
```

chgrp

El comando chgrp (**change group**: cambiar grupo en inglés) se utiliza para cambiar el grupo de un fichero. El grupo además de root, lo puede cambiar el propio dueño, siempre que pertenezca a ese grupo.

Sintaxis:

chgrp [opciones] <grupo> <ficheros>

Opción: -R en los directorios cambia el grupo recursivamente.

Ejemplos:

```
# chgrp clases foto1.png
```

En realidad, con el comando chown se pueden cambiar de una sólo vez el usuario y el grupo propietarios, pero sólo root cuenta con los permisos para poder hacer esto. Para cambiar también el grupo basta con añadirlo al usuario separado por el caracter dos puntos o simplemente un punto:

```
# chown morfeo:matrix foto1.png
# chown -R root.root datos/
# chown -R www-data: /var/www/website
# chown -R root. datos/
# chown :matrix foto1.png
```

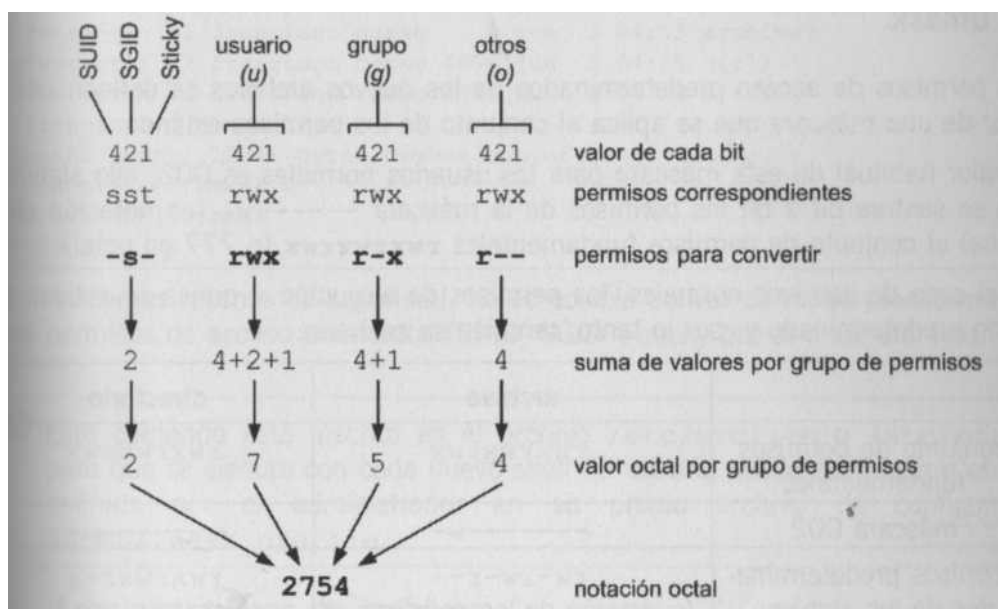
Cambiar permisos con Números (formato numérico)

Las nueve posiciones de permisos son en realidad un bit que o está encendido (mostrado con su letra correspondiente r, w o x) o está apagado (mostrado con un guión -). Cada permiso tienen asignado un valor, incluso cuando el permiso no está activo. La combinación de valores de cada grupo de usuarios (propietario-grupo-otros) forma un número octal (base 8, es decir, se utilizan 3 bits para representarlo y por tanto los permisos pueden tomar valores de 0 a 7), tenemos entonces:

- $r = 4$ (lectura) = 2^2
- $w = 2$ (escritura) = 2^1
- $x = 1$ (ejecucion) = 2^0
- $- = 0$ (sin permisos) = 0

Cuando asignamos los permisos utilizando números debemos tener en cuenta que primero se sumarán los valores y dicho resultado será el que se coloque, aquí una tabla que muestra dichos valores:

Permisos	Bits	Valor	Descripción
---	$0+0+0$	0	El valor cero significa que no se han asignado permisos
--x	$0+0+2^0$	1	sólo se ha asignado el de ejecución
-w-	$0+2^1+0$	2	sólo permiso de escritura
-wx	$0+2^1+2^0$	3	permisos de escritura y ejecución
r--	2^2+0+0	4	sólo permiso de lectura
r-x	2^2+0+2^0	5	permisos de lectura y ejecución
rw-	2^2+2^1+0	6	permisos de lectura y escritura
rwX	$2^2+2^1+2^0$	7	permisos: lectura, escritura y ejecución



Los permisos por números se asignan en grupos de 3, es decir, para el propietario-grupo-otros, no es factible asignar solo para uno o dos de ellos.

Ejemplos:

Permisos	Valor	Descripción
rw-----	600	Sólo el propietario tiene permisos de lectura y escritura.
rw-x--x--x	711	El propietario tiene los permisos de lectura, escritura y ejecución, el grupo y otros solo ejecución.
rwxr-xr-x	755	El propietario lectura, escritura y ejecución, el grupo y otros pueden leer y ejecutar el archivo.
rw-rw-rwx	777	El archivo puede ser leído, escrito y ejecutado por quien sea, ¡no es una buena elección!
r-----	400	Solo el propietario puede leer el archivo, pero ni el mismo puede modificarlo o ejecutarlo y por supuesto ni el grupo ni otros pueden hacer nada en él.
rw-r-----	640	El usuario propietario puede leer y escribir, el grupo puede leer el archivo y otros no pueden hacer nada.

Ejemplo	Descripción	Resultado
\$ touch foto2.png	creamos el archivo foto2.png por defecto con permisos 644	-rw-r--r--
\$ chmod 000 foto2.png	quitamos todos los permisos al archivo foto2.png	-----
\$ chmod 700 foto2.png	añadimos todos los permisos para el propietario	-rw-x-----

\$ chmod 710 foto2.png	añadimos el permiso de ejecución para el grupo	-rwx--x---
\$ chmod 714 foto2.png	añadimos el permiso de lectura para los otros usuarios	-rwx--xr--
\$ chmod 114 foto2.png	eliminamos los permisos de lectura y escritura para el propietario	---x--xr--
\$ chmod 444 foto2.png	establecemos como unico permiso de lectura para los 3 grupos	-r--r--r--
\$ chmod 555 foto2.png	establecemos los permisos de lectura y ejecución para los 3 grupos	-r-xr-xr-x
\$ chmod 000 foto2.png	quitamos todos los permisos	-----
\$ chmod 501 foto2.png	añadimos los permisos de lectura y ejecución al propietario y ejecución a otros	-r-x-----x
\$ chmod 550 foto2.png	añadimos permiso de lectura y ejecución al grupo y eliminamos permiso de ejecución a otros	-r-xr-x---
\$ chmod 770 foto2.png	añadimos permiso de escritura y ejecución al propietario y grupo, y eliminamos permiso de ejecución a otros	-rwxrwx---
\$ chmod 666 foto2.png	permite a cualquiera modificar el contenido e incluso eliminar el archivo	-rw-rw-rw-

Cambiar permisos con Carácteres (modo simbólico)

Otra manera popular de establecer los permisos de un archivo o directorio es a través de identificadores del bit (r,w, o x) de los permisos, como ya se vió anteriormente, pero ahora identificando además lo siguiente:

	Símbolo	Descripción
--	---------	-------------

Identidades	u	Es el usuario propietario del archivo o directorio
	g	Es el grupo al que pertenece el archivo o directorio
	o	Otros usuarios, el resto del mundo, ni el propietario ni su grupo
	a	Todo el mundo – propietario, grupo y otros
Permisos	r	Acceso de lectura
	w	Acceso de escritura
	x	Acceso de ejecución
Acciones	+	Añade los permisos
	-	Elimina los permisos
	=	el único permiso

Vamos a practicar con el comando `chmod`, para ello lo primero que haremos será crear el archivo `foto1.png` para ver los cambios de permisos, así que les recomiendo seguir la secuencia:

Ejemplo	Descripción	Resultado
\$ touch foto1.png	creamos el archivo foto1.png	-rw-r--r--
\$ chmod a-rwx foto1.png	quitamos todos los permisos al archivo foto1.png	-----
\$ chmod u+rwx foto1.png	añadimos todos los permisos para el propietario	rwX-----
\$ chmod g+x foto1.png	añadimos el permiso de ejecución para el grupo	rwX-x---
\$ chmod o+r foto1.png	añadimos el permiso de lectura para los otros usuarios	rwX-xr--

\$ chmod u-rw foto1.png	eliminamos los permisos de lectura y escritura para el propietario	--x--xr--
\$ chmod a=r foto1.png	establecemos como unico permiso de lectura para los 3 grupos	r--r--r--
\$ chmod a=rx foto1.png	establecemos los permisos de lectura y ejecución para los 3 grupos	r-xr-xr-x
\$ chmod a=- foto1.png	quitamos todos los permisos	-----
\$ chmod u+rx,o+x foto1.png	añadimos los permisos de lectura y ejecución al propietario y ejecución a otros	r-x--- x
\$ chmod g+rx,o-x foto1.png	añadimos permiso de lectura y ejecución al grupo y eliminamos permiso de ejecución a otros	r-xr-x---
\$ chmod ug+wx,o-x foto1.png	añadimos permiso de escritura y ejecución al propietario y grupo, y eliminamos permiso de ejecución a otros	rw-rwx---
\$ chmod a=rw foto1.png	permite a cualquiera modificar el contenido e incluso eliminar el archivo	rw-rw-rw-

Ubuntu | 11.- Gestión de permisos | 1.- chmod, c...



Gestión de permisos | 1.- chmod, chown y
chgrp: https://youtu.be/CU_DnPaP78U



Recuerda si cambiamos los permisos a un directorio y deseamos que estos permisos tengan efecto sobre todos sus subdirectorios y archivos sólo deberemos añadir la opción -R.

Ejemplo:

```
$ chmod -R 755 DIRECTORIO
```



Ejercicio Resuelto :: describir una máscara de permisos

Las nueve posiciones de permisos son en realidad un bit que o está encendido (mostrado con su letra correspondiente) o está apagado (mostrado con un guión -)...

así que, por ejemplo, ¿qué permisos obtenemos con un fichero que tuviera la siguiente máscara de permisos?:

```
rwxrw-r--
```

Mostrar retroalimentación



Ejercicio Resuelto :: permisos en formato numérico

Escribe la máscara de permisos utilizando rwx y su valor en formato numérico, por ejemplo: rw-rw-rw-666

1. Sólo el propietario tiene el derecho de leer y escribir.
2. Sólo el propietario tiene los permisos de leer y escribir; el grupo y los demás sólo pueden leer.
3. Sólo el propietario tiene los derechos de leer, escribir y ejecutar el archivo.
4. El propietario tiene los derechos de leer, escribir y ejecutar; el grupo y los demás sólo pueden leer y ejecutar.
5. El propietario tiene los derechos de lectura, escritura y ejecución; el grupo y los demás sólo pueden ejecutar.
6. Todo el mundo puede leer y escribir en el archivo. ¡No es una buena elección!
7. Todo el mundo puede leer, escribir y ejecutar. ¡Tampoco es buena elección!

Mostrar retroalimentación



Ejercicio Resuelto :: investigando el permiso de ejecución

La teoría nos dice que:

x: execute (ejecución): Si el **permiso de ejecución** está activo sobre un directorio significa que **el usuario podrá realizar otras funciones dentro de él mediante los otros permisos de lectura y escritura**, y si está activo sobre un **archivo** se podrá **ejecutarlo desde la línea de comandos**.

permiso	archivo	directorio
r	permiso de lectura del contenido del archivo	permiso para listar las entradas del directorio
w	permiso para modificar el contenido del archivo	permiso para modificar las entradas del directorio
x	permiso para ejecutar el archivo	permiso de acceso a las entradas del directorio

1. Prueba a asignar a un directorio únicamente el permiso de lectura (444) e intenta leer su contenido con un ls o leer alguno de sus ficheros de los cuales hayas apuntado previamente el nombre. Apunta qué has podido hacer y qué no.
2. Prueba a asignar a un directorio únicamente el permiso de ejecución (111) e intenta leer su contenido con un ls o leer alguno de sus ficheros de los cuales hayas apuntado previamente el nombre. Apunta qué has podido hacer y qué no.
3. Prueba a asignar a un directorio únicamente el permiso de lectura y ejecución (555) e intenta leer su contenido con un ls o leer alguno de sus ficheros de los cuales hayas apuntado previamente el nombre. Apunta qué has podido hacer y qué no.



Ejercicio Resuelto :: permisos con símbolos ugo

Completa la tabla

Actual	chmod	Resultado	Descripción
rw-----	a+x	rw-x--x	Agregar a todos (all) permisos de ejecución.
rw-x--x	go-x		
rw-r--r--	u-x,go-r		
rw-rw-rw-	u-x,go-rwx	rw-----	
r-----	a+r,u+w		
rw-r----		---rw---x	Al usuario se le eliminan lectura y escritura, al grupo se le agrega escritura y otros se le agrega ejecución.

4. Configuración de red

A continuación aprenderemos cómo configurar una interfaz de red (tarjeta de red) en Ubuntu con el objetivo de poder comprobar y solucionar posibles problemas de conectividad en un momento dado. Como ya sabes, la **dirección IP** de una interfaz de red (tarjeta o adaptador de red) puede ser establecida de forma **manual**, o de forma **automática** utilizando un servidor **DHCP** (acrónimo en inglés de *Dynamic Host Configuration Protocol* protocolo de configuración dinámica de máquina). El servicio DHCP asigna la dirección IP a la interfaz **así como toda la configuración necesaria: máscara, puerta de enlace, servidores de dns a utilizar**, etc. Por defecto, la configuración de la interfaz está establecida de forma automática, para facilitar la conexión del equipo con un router para salir a Internet.

Lo normal cuando vemos las conexiones del equipo por comando (ip addr o ifconfig) es que te encuentres con **dos dispositivos de red**:

- ❑ Interfaz de bucle local (lo).
- ❑ Interfaz Ethernet (ens# o eno# o enp#s# o eth# dependiendo del hardware y de la versión de Ubuntu).

La interfaz de **bucle local** está referida a nuestra propia máquina, y su dirección **IP siempre es 127.0.0.1**; es una dirección IP reservada que tienen todos los equipos que se utiliza para establecer conexiones entre los propios programas de los mismos.

La interfaz Ethernet es realmente la que se corresponde con la tarjeta de red instalada físicamente en tu equipo y a través de la cual, tienes conexión a la red cableada mediante el correspondiente cable de red.

Ficheros de configuración y reinicio del servicio de red

Los ficheros de configuración de red en las distribuciones basadas en Debian se guardan en **/etc/netplan/**. Hay que editarlos como root y guardar los cambios.

Después hay que reiniciar el servicio de red con el comando:

\$ sudo netplan apply #Para ver información de dicho reinicio recomiendo añadir la opción `--debug`

Configuración por defecto con NetworkManager en Ubuntu Desktop

En Ubuntu **Desktop** el fichero que configura la red debe estar situado en **/etc/netplan/** y es un **.yaml** o **.yml** utilizando NetworkManager como **gestor de red o renderer** por defecto; no obstante si por cualquier circunstancia no estuviese, copia algún ejemplo en [Internet](#) o esto:

```
network:
  version: 2
  renderer: NetworkManager
```

La ventaja de NetworkManager es que administra todos los dispositivos de red automáticamente, de manera que cualquier dispositivo Ethernet que añadamos en caliente, será reconocido y tendrá DHCP una vez que se detecte que hay señal en la tarjeta. Networkd, renderer por defecto en Ubuntu Server/Cloud no tiene esta cualidad.

Configuración de red automática (cliente de DHCP también llamado **dhclient**)

Si queremos configurar la red de manera automática -como la mayor parte de los dispositivos cuando estamos en nuestra casa- deberemos configurar en DHCP la interfaz de red de nuestro equipo o máquina virtual. De esta manera se nos dará una configuración de red dinámica o que se renueva a lo largo del tiempo. Si la interfaz de red ethernet se identifica como **enp0s3**, una configuración podría ser:

```
network:
  version: 2
  renderer: NetworkManager
  ethernets:
    enp0s3:
      dhcp4: true
```

Nota: Presta especial atención a los espacios: barra espaciadora o tabulador, no os recomiendo mezclarlos dado que la edición de ficheros yaml tiene la peculiaridad que no es lo mismo un espacio+tabulador que 2 espacios, y si un cierto parámetro debe estar a la altura de otro/s puede que netplan no lo coja. Por ejemplo esta configuración nos daría un error en línea 4 y la que está movida es la 6:

```
network:
  version: 2
  renderer: NetworkManager
```

```
ethernets:
  enp0s3:
    dhcp4: true
```

Configuración de red estática

Si queremos configurar IP dinámica en enp0s3 y estática en enp0s8 [por ejemplo](#):

```
network:
  version: 2
  renderer: NetworkManager
  ethernets:
    enp0s3:
      dhcp4: true
    enp0s8:
      dhcp4: no
      dhcp6: no
      addresses: [192.168.1.110/24 ]
      gateway4: 192.168.1.1
      nameservers:
        addresses: [8.8.4.4, 208.67.222.222]
```

Después de modificar el fichero anterior deberás ejecutar el siguiente comando para aplicar la nueva configuración:

\$ sudo netplan --debug apply

Cliente de DNS

El archivo de configuración de los servidores de nombres es **/etc/resolv.conf**, un contenido de ejemplo podría ser:

```
nameserver 8.8.4.4    # DNS de Google
nameserver 208.67.222.222  # DNS de OpenDNS
nameserver 1.1.1.1    # DNS de Cloudflare
```

Un cliente DNS irá preguntando a los servidores de nombres por orden de aparición en el fichero. Esto no ha cambiado, pero como desde versión 17.10 Ubuntu incorpora un servidor interno de DNS con caché llamado **systemd-resolved**, encontraréis que por defecto el fichero resolv.conf apunta por tanto a dicho servidor en la IP: 127.0.0.53. Para saber qué servidores DNS está usando realmente el sistema para resolver los nombres, puedes usar:

\$ resolvectl status

-> Current DNS Server es el servidor que está resolviendo los nombres de internet en ese momento

Proyecto iproute2 y comando ip

Si sabías algo de Linux, quizá te suenen los comandos ifconfig, route o netstat... pues bien, son comandos a extinguir dado que el proyecto net-tools que los sustentaba está parado desde hace años. Los comandos del nuevo proyecto iproute2 que viene por defecto ya en Debian y derivados, son comandos que se van a utilizar durante muchos años, tienen mucha funcionalidad y además aportan una serie de mejoras interesantes.

El comando ip es uno de ellos, permite la gestión de la red en tiempo real en el equipo, si queremos que nos muestre las interfaces:

\$ ip addr

\$ ip a

Si queremos ver la tabla de rutas o rutas hacia una IP concreta:

\$ ip route

\$ ip r

\$ sudo ip route get 8.8.8.8

Para más info: <https://ayudalinux.com/comando-ip/>

Comprobar funcionamiento de red

ping

Para comprobar que tenemos red, lo mejor es hacer un ping a un sitio de internet. Si falla, probaremos a poner una dirección IP en lugar de un nombre.

\$ ping 8.8.8.8 #en Linux está enviado paquetes hasta que no se para con Ctrl+C

\$ ping -c5 google.es #envía 5 paquetes al destino

ss

ss muestra las conexiones que se están usando en la tarjeta de red. Para saber los puertos abiertos en el sistema:

\$ ss -plant

-l muestra conexiones que están escuchando

-a muestra todas las conexiones

-n muestra ips en lugar del nombre

-t sólo muestra conexiones tcp

-p muestra el identificador del proceso que está usando el puerto

Cambiar el nombre del equipo en Ubuntu en 2 pasos sin reiniciar

Tal y como explico en este vídeo: <https://www.youtube.com/watch?v=SeYkx8cQMMk> para cambiar el nombre de un host en Ubuntu vamos a modificar el fichero **/etc/hosts**, en la línea donde asocia 127.0.1.1 al nombre del equipo, hay que sustituir el antiguo por el nuevo-nombre-del-host. Posteriormente hay que ejecutar

\$ sudo hostnamectl set-hostname nuevo-nombre-host

Búsquedas estáticas con /etc/hosts

Hemos hablado del fichero **/etc/hosts** porque este fichero se utiliza para resolver nombres por IPs de manera fija o estática. Si nuestro host pertenece a una red con dominio habría que copiar el FQDN o nombre completo de nuestro servidor en dicho fichero, por ejemplo, si nuestra máquina se llama ubuntu-vm, tiene la IP 192.168.13.128 y además pertenece a la red/dominio delarioja.org habría que copiar en el **/etc/hosts** la siguiente línea:

192.168.13.128 ubuntu-vm.delarioja.org ubuntu-vm

De esta manera tanto un ping a ubuntu-vm como a ubuntu-vm.delarioja.org haría que se emitiesen paquetes con IP destino 192.168.13.128.

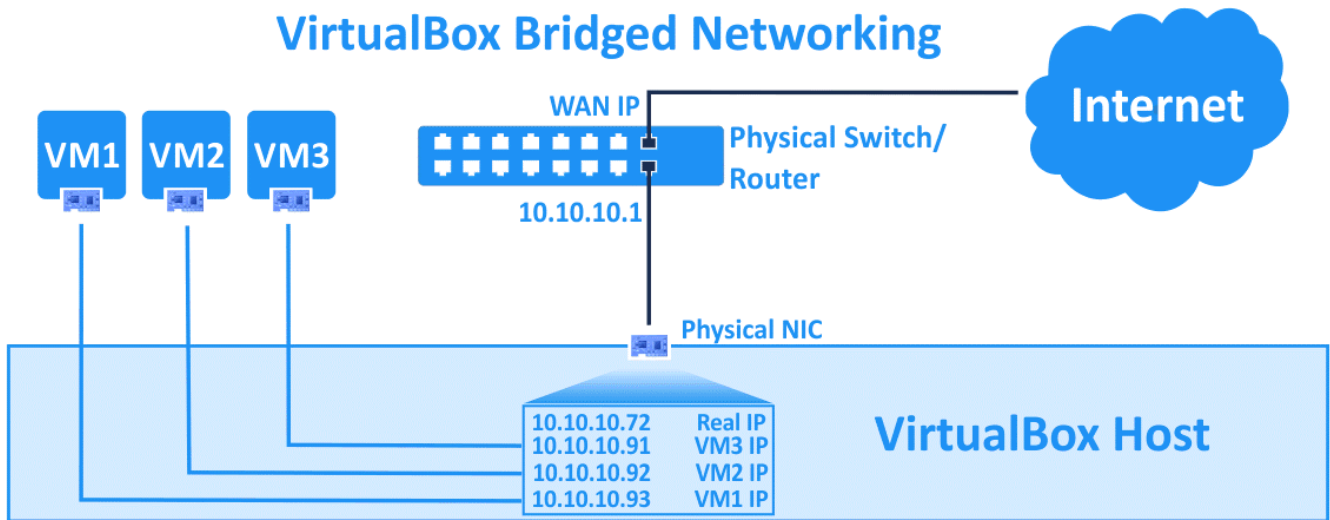
4.1. Configuración de red en virtualización

Entendiendo los Adaptadores de red virtuales de VirtualBox

Tanto VirtualBox como Docker proporcionan el uso de redes virtuales para que el usuario pueda agrupar máquinas virtuales o contenedores en redes independientes (dominios de difusión).

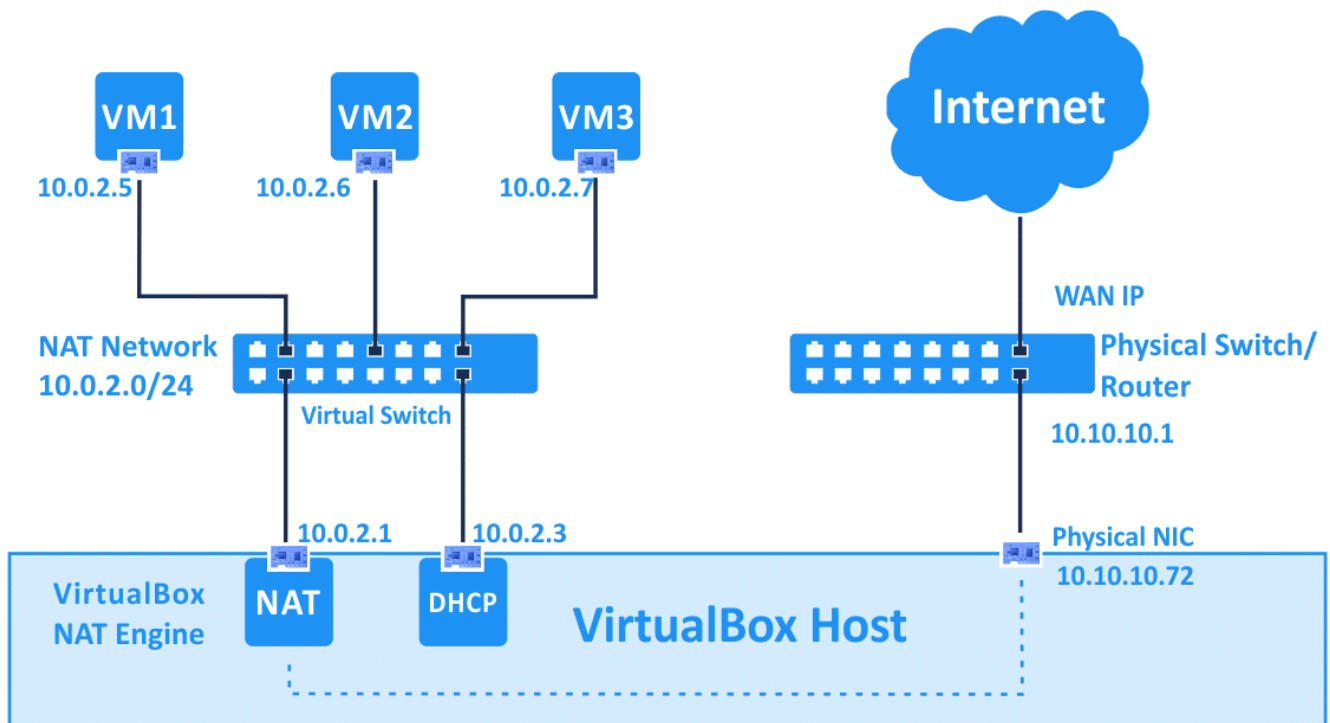
En el caso de VBox, por defecto, proporciona principalmente: Adaptador puente o bridged (host en docker), y Red NAT o NAT (bridge en docker). No obstante, también es posible utilizar otras configuraciones para conectar máquinas virtuales a nivel interno (red interna) entre sí.

Adaptador puente o bridged



Esta red virtual proporciona conectividad entre las máquinas virtuales, la máquina real y la red externa real. Las máquinas situadas en la red externa real pueden comunicarse directamente con las máquinas virtuales.

Red NAT o NAT



Esta red virtual proporciona conectividad entre las máquinas virtuales, la máquina real y la red externa real. En este tipo de red se realiza una traducción (PAT) de direcciones de red privadas a la dirección IP de la máquina real. Las direcciones privadas que utilizan las máquinas virtuales las proporciona un servidor DHCP.

En principio, las máquinas situadas en la red externa real no pueden comunicarse directamente con las máquinas virtuales debido al ocultamiento (PAT) de las direcciones IP privadas con la dirección IP de la



Reflexiona

1. Vete a la opción de Red de la configuración de tu máquina virtual y elige la opción de Adaptador puente. ¿Qué hace esta función? ¿Qué IP va a tener la máquina virtual?
 2. Arranca Ubuntu, y comprueba si tienes Internet. Luego cambia la configuración de red a NAT y comprueba si esta vez tienes Internet. ¿Qué hace esta función? ¿Qué IP va a tener la máquina virtual según las gráficas expuestas anteriormente?
- Para más información: [Nakivo](#)

4.2. Acceso remoto y SSH

El servidor de shell seguro o SSH (Secure SHell) es un servicio que permite que un usuario acceda de forma remota a un sistema Linux pero con la particularidad de que las comunicaciones entre el cliente y servidor viajan encriptadas desde el primer momento de forma que si un usuario malintencionado intercepta los paquetes de datos entre el cliente y el servidor, será muy difícil que pueda extraer la información. Para que un usuario se conecte a un sistema mediante ssh, deberá disponer de un cliente ssh.

Instalación del servidor ssh

Para instalar el servidor ssh debemos instalar mediante apt el paquete ssh que contiene tanto la aplicación servidora como la aplicación cliente:

// Instalación de servidor ssh

root@servidor:# apt install ssh

Los archivos de configuración son:

/etc/ssh/ssh_config: Archivo de configuración del cliente ssh

/etc/ssh/sshd_config: Archivo de configuración del servidor ssh (demonio)

Arranque y parada manual del servidor ssh

El servidor ssh, al igual que todos los servicios en Debian, dispone de un script de arranque y parada:

// Iniciar o Reiniciar el servidor ssh

sudo systemctl restart ssh

// Parar el servidor ssh

sudo systemctl stop ssh

Conexión al servidor mediante ssh

Para conectar desde un PC cliente al servidor mediante ssh, debemos ejecutar el comando ssh seguido del nombre ó dirección IP del servidor. La conexión se realizará con el mismo nombre de usuario que estemos utilizando en el PC cliente.

Ejemplo, supongamos que juan, desde el PC llamado cliente, quiere conectarse al servidor cuya IP es 192.168.1.239:

```
// Conexión por ssh
```

```
juan@cliente:~$ ssh 192.168.1.239
```

```
The authenticity of host '192.168.1.239 (192.168.1.239)' can't be established.
```

```
RSA key fingerprint is 51:70:3f:9c:ac:49:52:74:88:f5:45:a6:ae:f0:9c:8a.
```

```
Are you sure you want to continue connecting (yes/no)? yes
```

```
Warning: Permanently added '192.168.1.239' (RSA) to the list of known hosts.
```

```
Password: // Introducir contraseña de juan
juan@servidor:~$ // Ya estamos en el servidor
```

La primera vez que se conecte alguien desde dicho PC cliente, se instalará el certificado de autenticación del servidor, lo cual es normal si se trata de la primera vez. A la pregunta 'Are you sure you want to continue connecting (yes/no)?' debemos responder 'yes' ya que de lo contrario la comunicación se cortará. Si ya nos hemos conectado anteriormente otras veces y vuelve a realizar ésta pregunta, significa que alguien se está haciendo pasar por el servidor (nuestro servidor ha sido hackeado) o que se ha reconfigurado el servidor (cambio de nombre, IP, etc...)

Si deseamos conectarnos al servidor utilizando un nombre de usuario diferente, debemos incluir el nombre de usuario antes del nombre o IP del servidor y separado por una arroba '@'.

Ejemplo, supongamos que juan, desde el PC llamado cliente, quiere conectarse como miguel al servidor cuya IP es 192.168.1.239:

```
// Conexión por ssh como otro usuario
juan@cliente:~$ ssh miguel@192.168.1.239
Password: // Introducir contraseña de miguel en el servidor
miguel@servidor:~$ // Ya estamos en el servidor como miguel
```

Desde PCs con Windows es posible conectarse por ssh a servidores Linux mediante el programa Putty. Se trata de un cliente ssh para Windows que permite acceder en modo texto al sistema Linux desde sistemas Windows. Otro programa similar que, aunque menos popular, te recomiendo es [mobaxterm](#)

5. Procesos y logs

Como ya sabes, existen situaciones en las que puedes encontrarte con sistemas GNU/Linux sin entorno gráfico. En esos casos, no podrás utilizar herramientas tipo "Monitor del sistema" de Ubuntu similar al "Administrador de tareas" de Windows. Pero eso no significa que no puedas realizar tareas de monitorización. La diferencia, es que tendrás que utilizar comandos.

Comandos para controlar procesos

Linux es un SO multitarea, lo que quiere decir que varias tareas (procesos) están en ejecución a la vez. El *kernel* es el encargado de repartir los recursos (tiempo de CPU, memoria, etc) entre los distintos procesos. Cada proceso pertenece a un usuario y hereda de él sus permisos. Linux asigna a cada proceso un número de identificación llamado PID. Veamos los comandos relacionados.

ps

El comando ps (process status) lista los [procesos](#) con sus PID (Identificación del proceso) y nombre que se están ejecutando actualmente. Sin ninguna opción dará la lista de procesos que están corriendo desde la terminal donde se ejecuto el ps.

Sintaxis: ps [opciones]

Opciones:

- ❑ **-a** Muestra también procesos de otros usuarios.
- ❑ **-x** Muestra también procesos sin terminal de control (daemons y programas gráficos).
- ❑ **-u** Muestra más información de los procesos: usuario, fecha,... ; con **-u usuario** Muestra los procesos de un usuario específico.
- ❑ **-f** Muestra los procesos en forma de árbol indicando de forma gráfica quién es hijo de quién.

- ❑ **-l** Con todos los detalles (S: estado (S - Sleep, R - Running, T - Stopped), UID: Identificación del usuario que ha lanzado el proceso, PID: Identificación del proceso PPID: identificación del padre del proceso, valor de prioridad PR y nice NI, SZ: uso de memoria y CPU, etc.).

```

Archivo  Editar  Ver  Terminal  Ayuda  Herramientas  Ayuda
top - 10:03:50 up 1 day, 1:16, 2 users, load average: 0.62, 0.54, 0.51
Tasks: 176 total, 1 running, 175 sleeping, 0 stopped, 0 zombie
Cpu(s): 1.5%us, 1.5%sy, 0.0%ni, 97.1%id, 0.0%wa, 0.0%hi, 0.0%si, 0.0%st
Mem: 1992860k total, 1851208k used, 141652k free, 6432k buffers
Swap: 4000144k total, 272872k used, 3727272k free, 358960k cached

  PID USER      PR  NI  VIRT  RES  SHR  S  %CPU  %MEM    TIME+  COMMAND
 1151 root        20   0 201m 28m 8672 S   3  1.4 31:56.92 Xorg
10524 pepino      20   0 19224 1460 1064 R   1  0.1  0:00.13 top
    1 root        20   0 23700 1480  892 S   0  0.1  0:00.76 init
    2 root        20   0    0    0    0 S   0  0.0  0:00.01 kthreadd
    3 root        RT    0    0    0    0 S   0  0.0  0:00.01 migration/0
    4 root        20   0    0    0    0 S   0  0.0  0:00.82 ksoftirqd/0
    5 root        RT    0    0    0    0 S   0  0.0  0:00.00 watchdog/0
    6 root        RT    0    0    0    0 S   0  0.0  0:00.01 migration/1
    7 root        20   0    0    0    0 S   0  0.0  0:00.84 ksoftirqd/1
    8 root        RT    0    0    0    0 S   0  0.0  0:00.00 watchdog/1
    9 root        20   0    0    0    0 S   0  0.0  0:07.90 events/0
   10 root        20   0    0    0    0 S   0  0.0  0:01.16 events/1
   11 root        20   0    0    0    0 S   0  0.0  0:00.00 cpuset
   12 root        20   0    0    0    0 S   0  0.0  0:00.00 khelper
   13 root        20   0    0    0    0 S   0  0.0  0:00.00 netns
   14 root        20   0    0    0    0 S   0  0.0  0:00.00 async/mgr
   15 root        20   0    0    0    0 S   0  0.0  0:00.00 pm

```

Ejemplo:

Muestra todos los procesos en el sistema.

\$ ps -aux

Muestra todos los procesos de firefox en el sistema

\$ ps -aux | grep firefox

Se trata de un comando que nos proporciona información sobre los procesos.

Lo que podemos percibir en esta tabla de resultados es información, línea a línea, de cada proceso. Se muestran, como puede verse, los siguientes datos:

- ❑ **PID:** Cada proceso obtiene un número de identificación único, este es el PID.
- ❑ **USER.** Indica el uid (identificador del usuario real del proceso) del proceso, aunque vemos que el sistema lo sustituye por el nombre asociado a ese identificador de usuario.
- ❑ **%CPU.** Porcentaje de CPU (procesador) usado por el proceso.
- ❑ **%MEM.** Porcentaje de memoria real utilizada por el proceso.
- ❑ **VSZ.** Tamaño de la memoria virtual utilizada por el proceso.
- ❑ **RSS.** Tamaño de la memoria real del proceso en Kb.
- ❑ **TTY:** Es la terminal asociada al proceso, también puede ser otro dispositivo de entrada.
- ❑ **STAT o S.** Estado del proceso: S - Sleep, R - Running, T - Stopped
- ❑ **TIME:** El tiempo que el proceso ha estado en ejecución.
- ❑ **CMD:** El nombre del comando que está siendo ejecutado.

top

El comando top muestra los procesos ordenados por uso de CPU, además de otra información sobre el sistema, en tiempo real. Entre esa información puedes encontrar la cantidad total de memoria RAM utilizada, así como la cantidad de memoria virtual utilizada (esto lo puedes identificar con la palabra "swap"). Para salir de top con **q**, para matar un proceso con **k** y su PID.

Sintaxis: top [opciones]

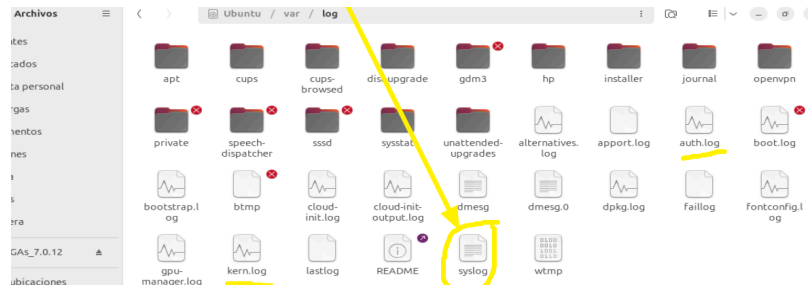
Ejemplo:

\$ top

Muestra todos los procesos en el sistema de forma continuada

Archivos de registro o logs del sistema

Otra de las herramientas básicas para evaluar el estado de un sistema en Windows es el visor de eventos. En GNU/Linux, los eventos importantes del sistema se almacenan en ficheros de *log*, y normalmente se encuentran en el directorio */var/log*.



Como puedes ver en la imagen, en este directorio existe una serie de archivos de texto, archivos comprimidos y carpetas. Todos ellos contienen información sobre sucesos (el concepto de "suceso" es equivalente al de "evento") asociados a diferentes aplicaciones y servicios. Puede que la información que obtienes en los ficheros de *log* te resulte abrumadora. Ten en cuenta que lo habitual es acudir a estos registros cuando buscas un error en concreto. Por ejemplo, si notas que en tu sistema no puedes arrancar el servicio *apache*, acudirás a mirar los archivos de *log* relacionados con dicho servicio y recorrerás su contenido en busca de errores.

Existen muchos motivos por los que se pueden generar mensajes. Entre los más frecuentes se encuentran

- ❑ los fallos del servidor: problema de hardware, fallo en un servicio, etc.
- ❑ de autenticación: el fichero */var/log/wtmp* registra los usuarios que se han logueado; a este fichero se accede mediante comandos; *lastlog*: indica la última vez que un usuario se logueó; *last*: muestra todos los accesos al sistema; *faillog*: busca logueos fallidos
- ❑ utilización de un servicio: petición de un cliente de una página web, etc.

La mayor parte de estos mensajes son reportados en el fichero */var/log/syslog* (en otros sistemas también en */var/log/messages*), pero si un servicio genera muchos mensajes lo normal es que sean escritos en un fichero o carpeta aparte como lo hace por ejemplo el servidor de correo */var/log/mail*.

El registro de todos los mensajes del sistema lo realiza el servicio *rsyslog*, el cual no es exclusivo de los servicios del sistema sino que nosotros también podemos registrar nuestros propios mensajes (por ejemplo para registrar avisos del cortafuegos *iptables* podríamos añadir a */etc/rsyslog.conf* la siguiente línea: *kern.warning /var/log/iptables.log*).

Sistema de ficheros */proc* - Varias estadísticas del kernel

El sistema de ficheros */proc* proporciona información detallada sobre los dispositivos hardware y otra información relacionada con el kernel de Linux. Ejemplos típicos podrían ser:

```
# cat /proc/cpuinfo
# cat /proc/meminfo
# cat /proc/zoneinfo
# cat /proc/mounts
```

o `cd /proc/pid-del-proceso` para saber los datos referentes a un proceso

6. Práctica "Conduciendo Linux"



Ejercicios obligatorios: "Gestión de Usuarios y Permisos"

Demuestra que ya sabes conducir un Linux resolviendo los ejercicios obligatorios que se proponen:

Basados en los comandos `adduser`, `deluser`, `passwd`, `usermod`, `addgroup`, `delgroup`, `su`, `sudo`, `chmod`, `chown`

En un curso de formación, queremos explorar diferentes maneras de que los alumnos entreguen sus tareas al profesor en un entorno Linux. Habrá un profesor (profesor), 4 alumnos (alumno1,...,alumno4) y 2 grupos de prácticas (grupo1 y grupo2; alumno1 y alumno2 pertenecen al grupo1, alumno3 y alumno4 al grupo2). **Solución. Resultado a enviar:**

- captura de la creación de los usuarios, los grupos y cómo se asocian los usuarios a los grupos

1. En la primera, habrá un directorio `/home/clase_linux` en el que los 4 alumnos linuxeros (alumno1, alumno2, alumno3 y alumno4) podrán crear y modificar sus ficheros (pista: crear un **grupo nuevo**). Además, los ficheros podrán ser leídos pero no modificados por el profesor y el resto de alumnos del instituto. **Comprueba estos permisos** pero antes, te recomiendo que ejecutes: `sudo chmod g+s /home/clase_linux`, investiga el uso de este comando y explica para qué va a servir. **Solución.**

Resultados a enviar:

- captura de la creación del grupo linuxeros y la asociación de los usuarios al mismo
- captura de la creación y configuración de la carpeta `clase_linux`: `mkdir`, `chmod`, `chown`
- captura del alumno1 haciendo un `cp /home/alumno1/documento.txt /home/clase_linux/` y del profesor haciendo `cat /home/clase_linux/documento.txt` pero no `cat /home/profesor/correccion-ej1.txt >> /home/clase_linux/documento.txt`

2. Para las prácticas en grupo, cada grupo dispondrá de un directorio `/home/grupo1`, `/home/grupo2` (deben crearse manualmente) en el que podrán escribir los miembros del grupo. Todos los alumnos pertenecientes a un grupo (alumno1 y alumno2 pertenecen al grupo1, alumno3 y alumno4 al grupo2) pueden escribir en el directorio del grupo (`/home/grupoX`), pero no pueden leer ni escribir en el directorio del otro grupo. Esos ficheros podrán ser vistos y modificados por el profesor. Comprueba estos permisos no sin antes ejecutar lo aprendido en el anterior ejercicio: `sudo chmod g+s ...`

Solución. Resultados a enviar:

- captura con la creación y configuración de los permisos de las carpetas de grupo: `mkdir`, `chmod`, `chown`;
- captura de cómo el alumno1 hace un: `cp /home/alumno1/documento.txt /home/grupo1/` pero no hace un: `cp /home/alumno1/documento.txt /home/grupo2/` ni tampoco un `ls -l /home/grupo2`
- captura de cómo el profesor puede ejecutar `cat /home/grupo1/documento.txt` y `ls -l /home/grupo2`

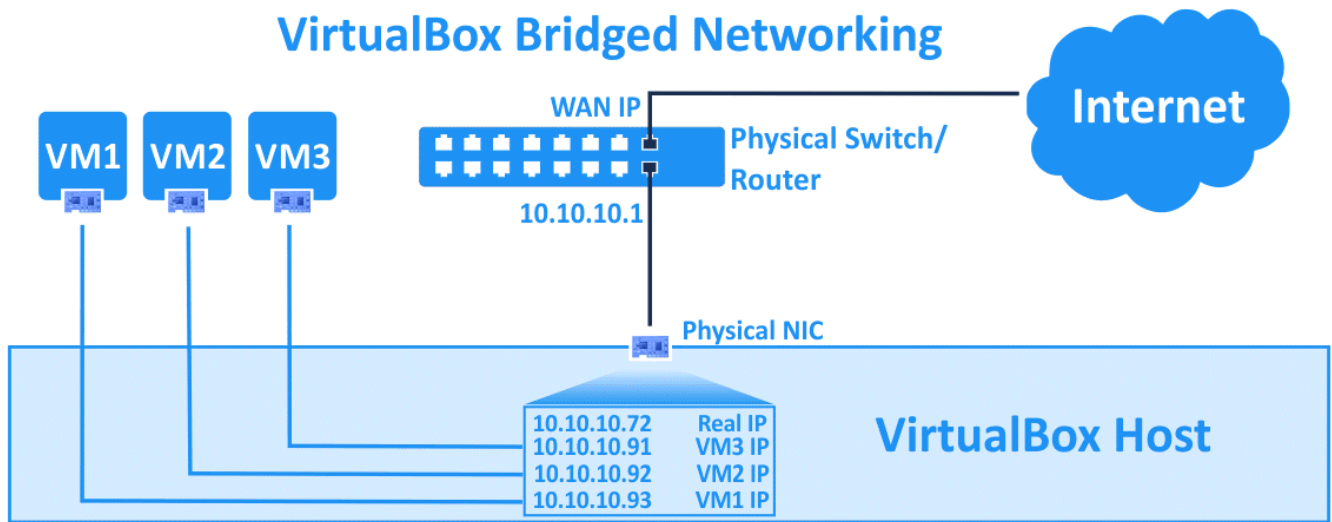


Ejercicios obligatorios: "Configuración de red"

3. Comprueba desde el icono de red de la barra de tareas de tu Ubuntu, la configuración de red que tienes asociada a tu tarjeta de red (IPv4, puerta de enlace, DNS). Después comprueba el contenido del fichero que hay en la carpeta `/etc/netplan/` y responde: ¿está configurado para recibir una configuración de red automática o tiene una configuración estática asignada? ¿qué pasará si desde VirtualBox cambiamos la configuración del adaptador de red de "NAT" a "Adaptador puente"?

Resultados a enviar: respuestas a las 2 preguntas. Para responder a la segunda pregunta puedes servirte de la siguiente imagen:

VirtualBox Bridged Networking



4. En VirtualBox, configura la tarjeta de red de tu Ubuntu como Adaptador puente. Posteriormente, espera unos segundos y ejecuta el comando **ip a** para ver si ha obtenido IP a través de algún servidor de DHCP de tu red (de tu router por ejemplo):

- En caso afirmativo demuestra la conectividad entre tu anfitrión y la máquina virtual mediante el comando **ping**
- En caso negativo sustituye el contenido del fichero existente en `/etc/netplan/` **por este otro** o establece una configuración estática coherente con la red física del equipo anfitrión (Physical NIC en la imagen anterior) para que puedan “verse”. Debe quedar así -suponiendo que la interfaz se llame `enp0s3` y que el router/gateway de tu red tenga la IP `192.168.1.1` y la tarjeta del SO anfitrión tenga por ejemplo la `192.168.1.126`:-

```
network:
  version: 2
  renderer: NetworkManager
  ethernet:
    enp0s3:
      dhcp4: no
      dhcp6: no
      addresses: [192.168.1.110/24]
      routes:
        - to: default
          via: 192.168.1.1
      nameservers:
        addresses: [8.8.4.4, 1.1.1.1]
```

Guarda y aplica con **sudo netplan apply** comprobando con **ip address** si se han aplicado los cambios. demuestra la conectividad entre tu anfitrión y la máquina virtual mediante el comando **ping**.

Resultados a enviar: captura del ping entre anfitrión y máquina virtual Ubuntu o viceversa



Ejercicios opcionales: puede entregarse uno, varios, todos o ninguno.

5. **Ejercicio OPCIONAL O EXTRA:** En una tercera manera de entrega de tareas, cada alumno va a configurar dentro de su directorio personal, una carpeta llamada profesor, es decir, si el usuario se llama alumnoX, creará la carpeta `/home/alumnoX/profesor`. En dicha carpeta el profesor va a poder

leer pero no modificar cualquier fichero que copie en la misma el alumno. El propio alumno tendrá acceso completo a la misma y el resto de usuarios no podrá hacer nada. Una vez tengas la carpeta ya creada con los permisos adecuados te recomiendo que antes de probar ejecutes: `sudo chmod g+s /home/alumnoX/profesor`, investiga el uso de este comando y explica para qué va a servir. Comprueba estos permisos. **Solución. Resultados a enviar:**

- captura de la creación y conf. de la carpeta: `mkdir, chmod y chown`;
- captura de cómo el profesor puede ejecutar un cat de un fichero contenido en la carpeta `/home/alumno1/profesor` pero no un:
`cp /home/profesor/correccion-ej1.txt /home/alumno1/profesor/`

Una manera de implementar los ejercicios anteriores en diferentes máquinas, sería agrupar los comandos en un fichero y lanzarlo/ejecutarlo en cada una de ellas. Por eso, se propone crear 2 script, uno que sirva para crear los usuarios/carpetas y otro para destruirlos: **solución**, empecemos por este último:

6. Crear un script (fichero de texto con los comandos requeridos) llamado `permisosBorra.sh` que elimine todos los usuarios, grupos, directorios y archivos creados en los 3 puntos anteriores. Dale permisos de ejecución y ejecútalo como root de la siguiente manera:

```
# chmod +x permisosBorra.sh
```

```
# ./permisosBorra.sh
```

o también

```
# bash permisosBorra.sh
```

7. Crear un script (fichero de texto con los comandos requeridos, uno por línea) llamado `permisosCrea.sh` que permita crear todo el proceso especificado en los 3 puntos iniciales. Dale permisos de ejecución y ejecútalo como root de la siguiente manera:

```
# ./permisosCrea.sh
```

Mostrar retroalimentación



Ejercicios opcionales: puede entregarse uno, varios, todos o ninguno.

- Realiza el tipo test de la unidad 2 hasta que saques más de un 8 sobre 10 en la calificación. Manda una captura de pantalla insertada en el documento/plantilla cuando lo tengas.
- Instala el servicio de SSH en tu máquina virtual Ubuntu y trata de accederlo desde tu Sistema Operativo anfitrión.

6.1. Evaluación de la tarea

¿Cómo valoro tu tarea? -> Rúbrica de la tarea

Ejercicio 1:

2 p.

Ejercicio 2:	2,5 p.
Ejercicio 3:	2 p.
Ejercicio 4:	2,5 p.
Bibliografía web y Valoración personal	0,5 + 0,5 puntos
Ejercicios Opcionales y Retos:	hasta +0.25 puntos extra cada uno
Ejercicios de Ampliación, Comentarios detallados:	hasta +1 punto extra cada uno

Preguntas autoevaluación



Autoevaluación Apartado 3

El usuario "usuario1" es el propietario de una determinada carpeta, y pertenece a un grupo llamado grupo1. Si quiere que únicamente tengan acceso a la carpeta los usuarios: usuario2 y usuario3, ¿Es totalmente obligatorio que todos los usuarios pertenezcan al mismo grupo?

- ☐ No es obligatorio solamente es opcional.
- ☐ Sí
- ☐ No, puesto que no tiene ninguna relación la asignación de permisos con la pertenencia al mismo grupo.



Autoevaluación Apartado 4

Relaciona los comandos con lo que muestran, escribiendo el número asociado a lo que hace el comando en el hueco correspondiente.

Comando	Relación	Resultado
ping	☐	1.- Muestra la IP.
ip addr	☐	2.- Muestra información sobre las conexiones de red.
mtr	☐	3.- Muestra el camino de un paquete.
ss	☐	4.- Comprueba la conexión.

Enviar



Autoevaluación Apartado 4

¿Qué comando te permite visualizar la configuración de red de las interfaces desde el terminal?

- ☐ ipconfig
- ☐ ip a
- ☐ ping
- ☐ ss

Créditos



Por *Fernando Barcina*, profesor del *Centro Integrado Público de Formación Profesional a Distancia de La Rioja*

Obra publicada con Licencia Creative Commons Reconocimiento Compartir igual 4.0